

Tsugite 継ぎ手

By Amir Ghadiri and Elisa van Klink

Custom emergency shelter
based on Japanese wood joints



Technical University Delft
AR3B012 CORE

Colophon

Tsugite 継ぎ手
Post-earthquake shelters,
based on japanese wood joints

AR2B012 CORE

10 november 2022

Amir Ghadiri
Elisa van Klink

Technical University of Delft
Master track Building technology
AR3B012 CORE

Contents

1 Introduction	4 - 5
2 Main problems	6
3 Main goals	7
4 The design	8 - 11
4.1 The design of the shelter	8 - 11
4.1.1 Japanese wood joints	8-9
4.1.2 Himişi	10
4.1.3 Materiality	11
4.1.4 Final result	11
5 Process and products	12 - 35
5.1 Flowchart	13
5.1.1 An overview of the process	12
5.1.2 Section 1: User Interface and data	12
5.1.3 Section 2: Interactive Design and User	12
5.1.4 Section 3: Computational Design	12
5.1.5 Section 4: Structural Analysis	12
5.2 From drawing to shelter	14 - 15
5.2.1 How to translate a drawing to a grid	14 - 15
5.3 The design of the interface	16 -21
5.2.2 The interface design	16 -17
5.2.3 What happens behind the scenes?	18 - 21
5.4 Model generation	22 -29
5.4.1 How to communicate with rhino, GH	22 - 23
5.4.1 Getting the right plan	22 - 23
5.4.2 Creating a grid	24 - 25
5.4.3 Generating the joints	26 - 27
5.4.3 Generating beams and columns	28 -29
5.5 Structural design	30 - 33
5.5.1 The idea	30
5.5.2 Our approach - Ansys	30
5.5.3 Our approach - Karamba 3D	31 - 33
5.6 Getting ready for production	34 - 35
5.5.1 Delivering a pysical shelter	34
5.5.1 A scale model	35
6 Last thoughts	36 - 37
6.1 Conclusions	36 - 37
6.1.1 The final product	36
6.1.2 Reflection	36
6.1.3 Future possibilities	37
7 References	38 - 39
7.1 Literature	38 - 39
7.2 Used libraries, plugins and other apps	39
8 Appendix	40 - 51
8.1 Analysis on a 2 by 3 shelter in Ansys	40 - 47
8.2 Photos of the exhibition	48 - 49
8.3 Posters	50 - 51

1 Introduction

1.1 Introducing Tsugite

1.1.1 Meet the developers

Introducing the dynamic duo, Elisa van Klink (on the left) and Amir Ghadiri (on the right), the minds behind this project named “Tsugite.” Fueled by a shared passion for innovation and a commitment to addressing critical challenges, we stand at the forefront of a project meaning to design custom shelters specifically tailored for earthquake-prone regions. Join us on a journey through their collaborative efforts as we navigate the intricate landscape of seismic architecture, pushing the boundaries of conventional design to create safer and more adaptive living spaces.

1.1.2 What is our project about?

In response to the challenges posed by earthquakes in shelter design, our project embarked on a journey to develop a solution leveraging computational design and Python coding. At the heart of our innovation lies the concept of interlocking joints and structures, chosen for their simplicity in assembly, ease of transportation due to their compact, connectable parts, and suitability for rapid shelter construction post-earthquake. Hence the name Tsugite, which directly translates from Japanese to joinery. These structures, requiring no special equipment for assembly, also hold potential for future expansion into permanent architectural elements.

This endeavor builds upon our previous experiences in ‘Informatics’, particularly our exploration of Japanese interlocking joints. A significant insight from this past project was the realization of the ad-hoc approach traditionally employed in determining the size and geometry of structural elements. To address this gap, we introduced a set of defined parameters for the size and geometry of interlocking joints, enabling us to optimize the structure more effectively. These parameters are integral to our computational approach, ensuring both structural integrity and adaptability.

We also developed a user-friendly interface that allows individuals to design their own shelters. These designs are transformed into CNC-milled timber pieces, facilitating straightforward assembly into the desired structure. Our project introduces groundbreaking features, such as integrating braces within the structure to boost lateral stability and incorporating interlocking support beams to reduce deflection. Through these innovations, our project not only aims to provide immediate, efficient shelter solutions in the aftermath of earthquakes but also aspires to revolutionize sustainable architectural practices in seismically active regions.

1.1.2 The structure of the report

This report will delineate our project in five chapters, commencing with an exploration of the prevailing challenges in existing shelter design identifying the specific issues we aim to address in the following chapter. Subsequently, we will delve into our design decisions, presenting the outcomes of the system we developed, along with the implementation details of various wood joinery methods. The following chapter will document the application development process, elucidating the steps involved in generating custom shelter structures and conducting structural analyses. Finally, we will reflect on the overall process and articulate future plans for the project.



Figure 1: Photo of the installation at the presentation
Left Elisa van Klink, right Amir Ghadiri
Source: Amir Ghadiri

2 Main problems

2.1 What are the current struggles in emergency shelters?

2.1 A dire situation

In the aftermath of a major disaster like earthquakes, homes are often left damaged or in ruins, leaving a significant portion of the population without shelter.

For example, the 1999 Marmara earthquake in Turkey, registering a magnitude of 7.6 on the Richter scale, resulted in approximately 17,000 fatalities, nearly 50,000 injuries, and around 500,000 individuals displaced from their homes. Aftershocks from this seismic event persisted for several months, with the most significant occurring in Düzce, measuring 7.2 in magnitude (Kılıcı et al., 2015).

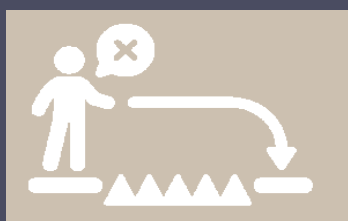
In the article of Kim et al. (2021) they mention that for the recovery of the people mentally the habitability of a shelter should be taken into account. For example they mentioned a 5.4 magnitude earthquake that hit Pohang, South Korea in November 2017, left nearly 1300 people homeless. Evacuees were accommodated in a hastily prepared gymnasium shelter due to ongoing aftershocks. Although tents were provided for privacy, they proved inadequate, being too small and unable to block external noise resulting in poor living conditions and mental health of the occupants. And you may think that this was only for a few weeks but this process took over 2 years due to the many aftershocks (Kim et al., 2021).

This is an example of how shelter as tents for the first few days are often quickly provided, but problems arise in the period between them transitioning from that tent to a permanent housing situation.

2.1 The main struggles

So what are the main points we wanted to tackle for a design for emergency shelter in this crucial time period, transitioning people over from emergency shelter to temporary sheltering to eventually a home?

- ♦ One of the problems is that they often have **no access to even the simplest of tools** let alone specialized ones. So you would still need a shelter that's quickly built but for example can withstand aftershocks and will provide a more habitable space than a simple tent.
- ♦ Moreover since most of the area is in ruins, the area's are often **hard to reach** with big vehicles like cranes, buses or trucks, making it quite hard to deliver certain materials, or construct big elements.
- ♦ One more thing to take into consideration is the **amount of people in a household** in the area. For example in Europe the average household is around 3.1, but in African countries this can go up to six or seven (Statista, 2022 and UN, 2019). Secondly, even within a neighborhood the differences can be great. There could be just two people living in an apartment, and in the same area there could be a household where the grandparents are with their children and their grandchildren totalling the amount of people living there to five, six or more. So creating a shelter that can be customized to the needs of the user, by example looking at household size would also be preferable.



3 Main Goals

3.1 How do we think we are going to solve the problems?

3.1.1 Our aim

Our aim lies with designing emergency shelters that cater to the needs of disaster-affected populations during the transitional period between immediate response and long-term **recovery**.

What do we aim to achieve with the new design for emergency shelter?

1. Quick construction and easy assembly:

- ◇ Develop a shelter that can be quickly assembled using minimal tools and manpower.
- ◇ Considering designs that are intuitive and don't require specialized skills for construction.
- ◇ Include clear, simple instructions for assembly.
- ◇ Structurally sound enough to withstand aftershocks

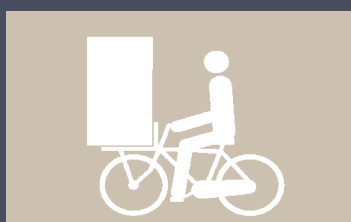
2. Accessibility and Transportation:

- ◇ Design components that are compact and lightweight for easy transportation, especially in areas where large vehicles may have difficulty reaching.
- ◇ Explore modular designs that can be transported more efficiently.

3. Adaptability to Varied Household Sizes:

- ◇ Create shelters that can be easily modified or expanded to accommodate varying household sizes.
- ◇ Consider adjustable partitioning or modular additions to cater to larger families.

In summary, our project seeks to develop a swift and uncomplicated emergency shelter solution. Users can customize it through a user-friendly interface that is accessible to anyone. Additionally, the assembly process is designed to be straightforward, requiring minimal tools and accompanied by clear instructions.



4 The design

4.1 The design of the shelter

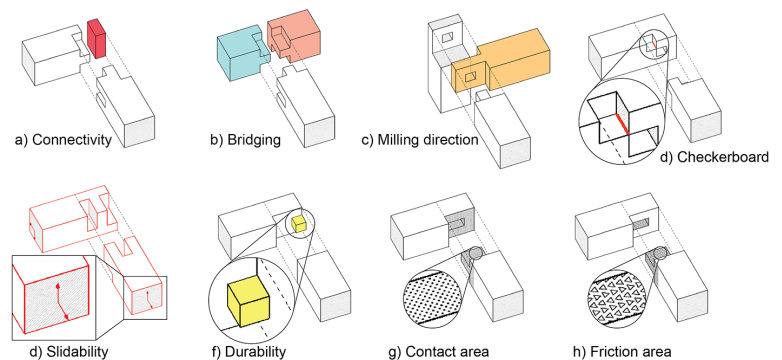
4.1.1 Japanese wood joints

Since we wanted to create something that was easily assembled and could withstand aftershocks we started by looking into different ways of construction. Eventually we came across the method of Tsugite. Meaning Joinery in Japanese. Which can be defined as a traditional Japanese wood joinery method that uses no bolts, glue nails or any other material than just the connection of the wood pieces (Lily, 2018).

To be exact: "Tsugite. In the processing of wood, when two materials are in contact with each other, the members are connected so that they are in a straight line." (ThemeGrill, 2021).

This is when we found the Application made by students of the University of Tokyo called Tsugite. In the app people were able to design their own joint and get live feedback on your creation on the subjects below. Inspiring us to do something similar but then for the entire system of an emergency shelter (Marialarsson, 2020).

Figure 2: Tsugite examples
Source: Marialarsson. (2020). GitHub - marialarsson/tsugite: This is the repository of a Tsugite. It is a research prototype of an interactive software that supports the design and fabrication of wood joints. GitHub. <https://github.com/marialarsson/tsugite>



And looking further into Japanese wood joinery like Tsugite we found there were many more methods like Tsugite that join wood together without the need of other materials or connectors.

For example we found the project of the GC Prosth Museum Research Center by Kengo Kuma & Associates. They used a joint type called Chidori. A Japanese six-pieces wooden puzzle inspired from traditional dougong suggesting a relatively rigid yet elastic post and beam assembly. Chidori have uniquely shaped joints, allowing extension through simple twisting motions, eliminating the need for nails or metal fittings (King, 2021 and Potvin et al., 2013).

Figure 3 and 4: Museum research center, and example of a how a chidori joint works
Source: King, V. (2021, March 3). GC Prosth Museum Research Center / Kengo Kuma & Associates. ArchDaily. <https://www.archdaily.com/199442/gc-prosth-museum-research-center-kengo-kuma-associates>



Later we came across a paper by Potvin et al. (2013). This was a project on Adaptive Architecture that led to a fully adaptable three-story high wooden structure based on two types of Japanese joints. The traditional Dougong joint assembly showing the progressive interlocking superposition of bowed shape gongs on a dou block. And a 'chidori', Japanese six-pieces wooden puzzle inspired from traditional dougong suggesting a relatively rigid yet elastic post and beam assembly.

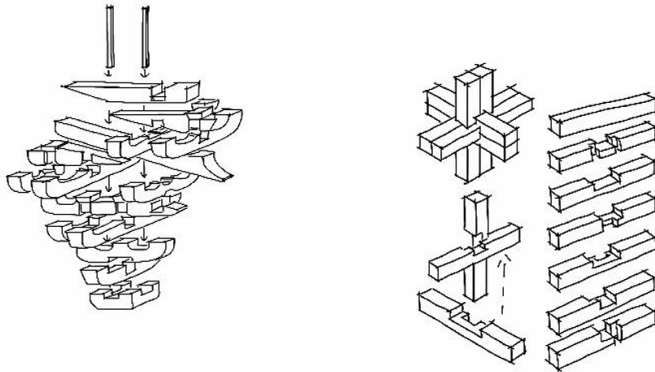


Figure 5 and 6: Dougong joint (left), Chidori joint (right)
Source: Potvin, A., Demers, C., & Dumontier, C. (2013). Wooden Adaptive Architecture System (WAAS). ResearchGate. <https://doi.org/10.13140/2.1.3295.3924>

The system they created allowed the easy maneuverability by the occupants of walls and floors to adapt the space to their environmental and functional needs. It consists of eighteen 6'x 6' (~1.8 x 1.8 meter) cells that can be entirely reconfigured with movable planes in x, y, and z directions.

The WAAS "Wooden Adaptive Architecture System", apart from providing fully adaptable space configurations, can be easily deconstructed, transported, and reassembled in totally new building shapes. The system was ultimately made of 2000 small prefabricated sticks measuring as little as 1 3/4" x 1 3/4" x 24" (~50 x 50 x 600 mm).

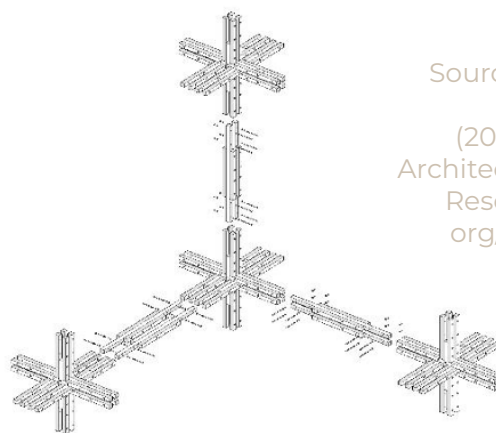
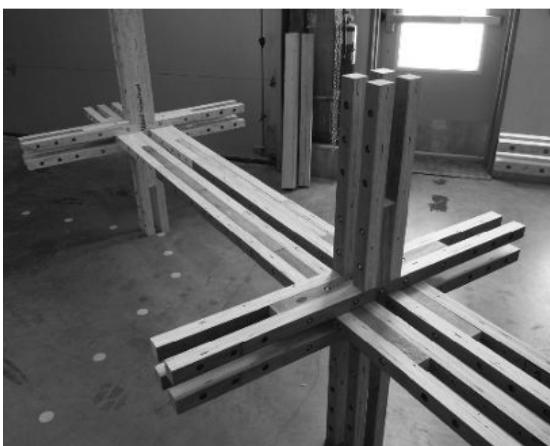


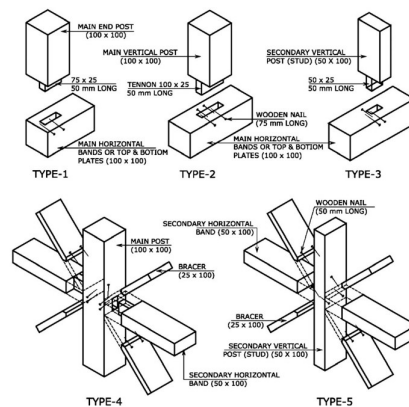
Figure 7 and 8: The WAAS system
Source: Potvin, A., Demers, C., & Dumontier, C. (2013). Wooden Adaptive Architecture System (WAAS). ResearchGate. <https://doi.org/10.13140/2.1.3295.3924>

This project has been a profound source of inspiration for our team, influencing our decision to adopt a similar system. Our approach emphasizes empowering individuals to craft their own adaptable 'grid,' eliminating the constraints of a predefined cell structure. In our design, wooden components dynamically adjust to varied situations and loads, offering custom dimensions tailored to each unique case. Furthermore, we introduce bracing elements to enhance resilience against seismic activity. As well as opting for a design that would not need the bolts like they used in this example but opting for something more close to the actual Japanese wood joints.

4.1.2 Himiş

The system above also lacked bracing which would be necessary if it were to withstand seismic activity. Or at least have some kind of protection against it. While searching for examples we stumbled upon a traditional Turkish construction technique called Himiş. Though the technique has been used all over the world with different names as Dhajji Dewari in Azia or Gaiola Pombalina in Portugal. Himiş uses timber frames that are highly subdivided, filled with either brick or stone (see figure 9 and 10). The primary structural elements are generally thinner than those of their European counterparts, and the secondary subdivisions are made of board-sized short pieces nailed to the primary elements. The subdivision patterns can vary from stiff cross-bracings to ever more ductile zig-zag, chevron, fishtail, or random assemblies.

Figure 9 and 10: Joint of the frame elements (left)
Example of himiş (right)
Source: Ali, Q., Ahmad, N., Ashraf, M., Rashid, M. M., & Schacher, T. (2017). Shake table tests on Single-Story Dhajji Dewari traditional buildings. *International Journal of Architectural Heritage*, 1–14. <https://doi.org/10.1080/15583058.2017.1338789> - Dhajji Dewari. (n.d.). <https://www.world-housing.net/tutorials/other/dhajji-dewari>



These structures have performed well in earthquakes throughout history due to a number of factors, particularly the high strength-to-weight ratio of timber as a material, its enhanced strength under short term loading, and the ability to dissipate large amounts of energy without collapsing. One of the few examples listed in the paper of Gülkan & Langenbach (2021) is shown below. Where a Himiş building on the left is still standing strong after an earthquake in the area. (Gülkan & Langenbach, 2021, Usta & Bozdağ, 2022, Ali et al., 2017).



Figure 11: A view of a Himiş (left) and a 4 story RC building
Source: Gülkan, P., & Langenbach, R. (2021). *Traditional timber-laced masonry construction in Turkey known as himiş*. In Elsevier eBooks (pp. 61–97). <https://doi.org/10.1016/b978-0-12-821087-1.00015-6>

This is why we also opted for a design using smaller frames and bracing each frame like the examples of the himiş so our structure would be safe against seismic activities.

4.1.3 Materiality

As for materials, it's probably pretty clear that we choose timber. For both the structural elements as the panels that would make up the walls. It's cheaper than for example concrete and steel and a whole lot easier to work with if you don't have any specialized tools (Homes, 2022 and Megan, 2023).

Furthermore it is a lot stronger for construction than the usual materials used to make tents. And it has the benefit that when working with a wooden structure it can easily be adapted to something more permanent, easing the transition from tent to a house.

There are many types of wood available. And for this course we decided to go for two options, a cheaper more pliable softwood, Pine and a stronger but more expansive type of Hardwood, Oak since these two species were mostly found in Turkey and our project started out there (Zengin et al., 2013). Though In the future we might want to opt for something where the types of wood available would be based on your location and what is mostly available.

4.1.3 Final result

With all of the above in mind we came up with our own design system that came to look like the pictures below. With 7 sets of joints, 8 types of beams, 1 type of column and 2 types of bracing as seen in the pictures below.

The joints are based on chidori. The bracing as well as the divided frames or grid based on himiş. The size of the grid (1 x 1 x 1 meters) was chosen since this was an easy number to calculate with as well as the fact people would now still be able to easily climb on top of each cell to construct the shelter as well as the fact that the themselves then wouldn't be to big so they are transported easily in not to big vehicles or carried and moved in place.

With these blocks we want to be able to give the people in need of shelter the option to fully customize their own based on something as simple as a drawing.

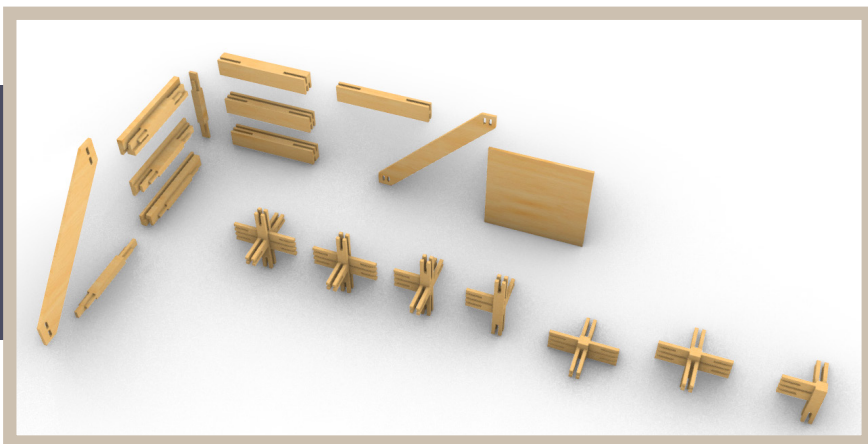


Figure 12 and 13: All components of the shelter (left) an example of one shelter (right)
Source: Own rhino/ enscape render

5 Process and products

5.1 Flowchart

5.1.1 An overview of the process

How would we go from a drawing to a shelter? In the flowchart to the right here the entire process is drawn out (for a better view see the miro board or appendix). We started out in python creating a canvas and interface for the user to fill in their information and draw a plan. From there we get a list of points (the grid) that will be send over to grasshopper rhino, through the use of a csv file, points.csv and a gh-python component that receives the points From there a first version of the structure is generated, analyzed and the optimal cross section is then led to the final component that generates the structure.

5.1.2 Section 1: User Interface Development and Data Collection

The process begins with a user interface, this serves as the primary point of interaction for users to input their requirements. On this interface, a structured form is presented, soliciting essential details such as the user's Name, Email, Address, Phone Number, as well as the intended amount of people the shelter will serve, and the number of stories desired. The form is designed to, ensure the correct data type is collected for each field. This step is critical as it captures the bespoke requirements that the computational model will later need to satisfy. With all information filled in the user is allowed to proceed to submit the data. The system initiates a validation protocol to verify the completeness and accuracy of the information provided. Should any discrepancies arise, the user is alerted and prompted to correct the entries before proceeding.

5.1.3 Section 2: Interactive Design and User Feedback Loop

Following data submission, the user is directed to a drawing interface, which features a digital canvas overlaid with a grid. This canvas is the creative workspace where users can visually draft the layout of their shelter using simple mouse interactions to place squares on the grid, which symbolize different parts of the shelter's plan. The drawing tool captures the coordinates of the user's mouse input, translating clicks and movements into square placements, thereby crafting a visual representation of the shelter's floor plan. This process is both intuitive and precise, affording users the flexibility to design freely within the constraints of the grid. Furthermore an iterative design loop is in place, allowing users to review and adjust their design. If the user decides the result is not satisfactory, they can utilize the 'clear canvas' function to erase the design and start anew, ensuring that the final product is to their liking. Once the user is content with the draft, they activate the render process by clicking the 'render button'. The system then performs a secondary validation, ensuring the design aligns with the spatial requirements calculated based on the number of people it is meant to house, using a formula that multiplies the expected number of occupants by a predefined spatial standard.

5.1.4 Section 3: Computational Design and Model Generation (see grasshopper block)

The design, now in a digital format, is processed to extract the center points of the drawn squares. These points are crucial as they serve as the foundation for translating the 2D plan into a three-dimensional grid, which will simulate the physical structure of the shelter. The processing stage extends into the use of Grasshopper, a visual programming platform integrated with Rhino 3D, which allows for sophisticated manipulation and arraying of the points to create a complex 3D line grid. This grid is the skeletal framework of the model, representing beams and other structural components. The generated 3D grid undergoes a series of checks to ensure that the lengths of beams and supports fall within practical and manufacturable limits, adapting the design to fit real-world constraints.

5.1.5 Section 4: Structural Analysis, Optimization, and Final Output

At this juncture, the model is subjected to an in-depth structural analysis facilitated by Karamba3D, a parametric structural engineering tool that evaluates the model against various loads and material properties to ensure its structural integrity. The tool performs optimization calculations, fine-tuning the cross-sectional areas and bracing of the structure to achieve maximum efficiency and stability without excess material use. A compliance verification process is implemented, assessing whether the optimized model adheres to building regulations, including deflection limits, to guarantee safety and habitability. Upon successful verification, the system finalizes the design, making the assembly instructions available for download. The user is also presented with a rendered visualization of the final structure on the interface, providing a realistic preview of the completed shelter. The last stage of the process involves preparing the individual components of the model for production. These parts are then fabricated, typically via CNC milling or other automated methods, and dispatched to the user for assembly, culminating in the realization of a custom-designed, earthquake-resistant shelter.

Section 1

Section 2

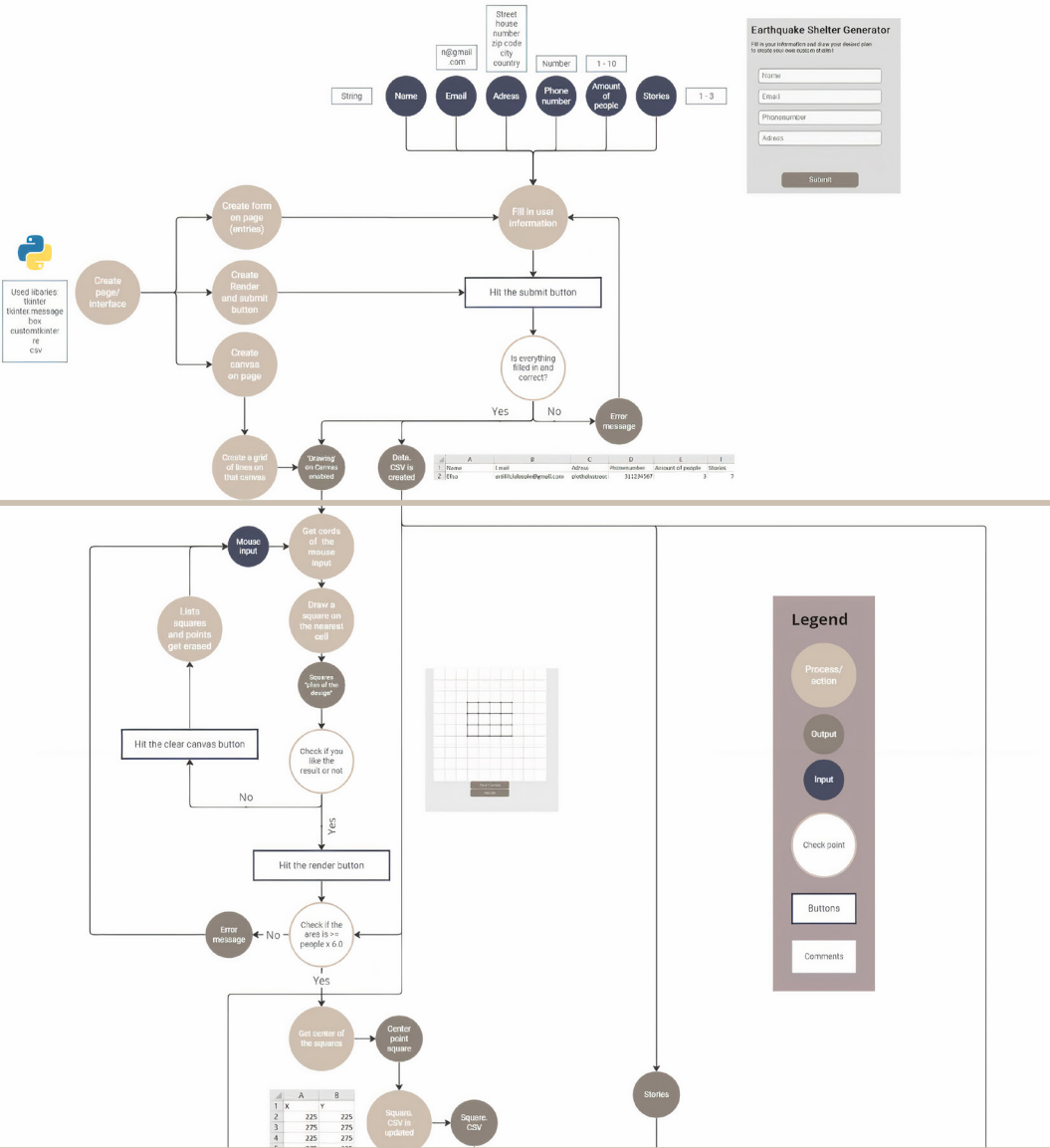


Figure 14: Flow chart of the entire process
Source: Our own design made with Miro
To see a high resolution version see miro board
<https://miro.com/welcome/>

Karamba
(plugin grasshopper)

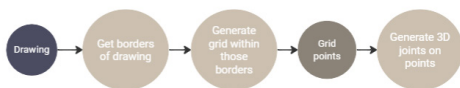
5.2 From drawing to shelter

5.2.1 How to translate a drawing to a grid

One of the main goals is to create a simple interface that anyone could use, and we thought it was a nice idea for people to sketch their own plan.

And as said in the previous chapter our design would consist of a number of cells all being 1 by 1 meters. On top of that we also had a script that would generate the joints we designed in 3D with python. How we did this will be more thoroughly discussed in the next paragraph, but simply said we did this by generating cuboids in a 3D space from a center point.

So the challenge lay in the fact that we needed to go from a drawing to a grid.



At first we tried the option of going from a sketch they draw on physical paper, upload it to the website and then from there generate a plan (see `picturetogrid.py`) however this seemed quite a lot of work for the user and they would need to make a good scan or picture of the plan for this option to work, which could be a hard thing to do in their circumstances so we quickly moved on to creating a app or interface of sorts that would let people draw on a canvas.

Therefore we decided that Tkinter would be a nice library to use for generating such a canvas, as this library also allowed us to create an interface around it where we could add buttons, inputs etc. In the snippets below you can see the three sets of code that make this possible.

- ◇ Starting with creating lists to save coordinates, a canvas 400 x 400 and the libraries (numpy for calculations and help creating a grid with the received cords)
- ◇ The middle block creates 3 definitions `start_draw`, `end_draw`, and `draw` so a line is drawn on canvas when someone clicks and holds the mouse button, and stops when released.
- ◇ The bottom one generates the grid (in this instance 20 by 20) within the area that it received of the most outer coordinates that takes up the space of the drawing. (400 being the max for x and y, min 0 for both as the canvas is 400 by 400)

Setting the variables, canvas and libraries

```
import tkinter as tk
import numpy as np

# Create a Tkinter window
root = tk.Tk()
root.title("Drawing Pad")

# Create a canvas for drawing
canvas = tk.Canvas(root, width=400, height=400, bg="white")
canvas.pack()

# Initialize variables for drawing
drawing = False
coords = []
```

Draw event

```
# Function to handle mouse button press (start drawing)
def start_draw(event):
    global drawing, coords
    drawing = True
    coords = []

# Function to handle mouse button release (end drawing)
def end_draw(event):
    global drawing, coords
    drawing = False
    coords.append((event.x, event.y))
    generate_grid()

# Function to handle mouse motion (drawing)
def draw(event):
    global coords
    if drawing:
        coords.append((event.x, event.y))
        canvas.create_line(coords[-2], coords[-1], width=2, fill="black")

# Bind the mouse events to their respective functions
canvas.bind("<ButtonPress-1>", start_draw)
canvas.bind("<ButtonRelease-1>", end_draw)
canvas.bind("<B1-Motion>", draw)
```

Grid generation within boundary

```
# Function to generate the grid within the drawn boundaries
def generate_grid():
    global coords
    if len(coords) > 1:
        x_coords, y_coords = zip(*coords)
        xmin, ymin, xmax, ymax = min(x_coords), min(y_coords), max(x_coords), max(y_coords)

        # Calculate the spacing based on your specific requirements
        spacing = 20 # Example spacing

        x_points = np.arange(xmin, xmax, spacing)
        y_points = np.arange(ymin, ymax, spacing)

        xx, yy = np.meshgrid(x_points, y_points)
        grid_points = np.column_stack((xx.flatten(), yy.flatten()))

        # Draw the grid points
        for x, y in grid_points:
            canvas.create_oval(x-1, y-1, x+1, y+1, fill="blue")
```

Result

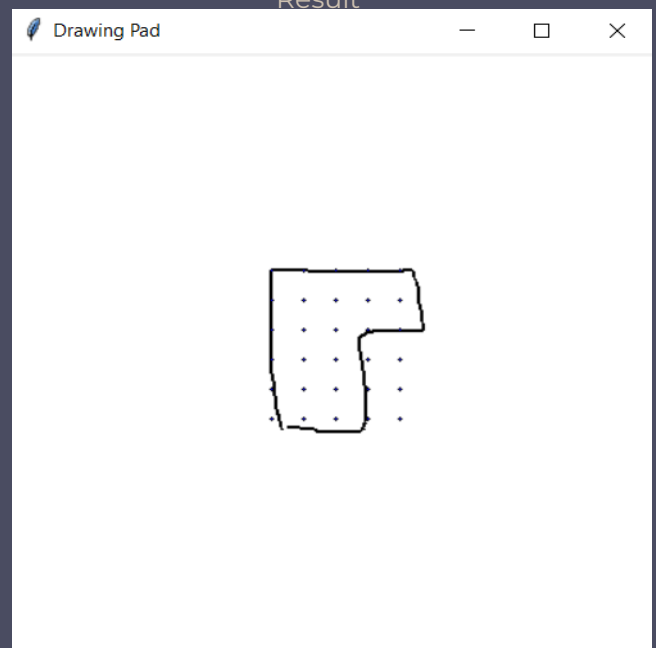


Figure 15, 16 and 17:

Example of first canvas (Draw.py)
Source: Own code, see <https://github.com/ArtiApple66/Earthquake-Shelter> (github repository)

This however came with the problem that the grid would always generate in a square or rectangle, not exactly following the shape drawn on canvas. And the way people would be able to draw is also quite messy, if you're just a bit off new points are added to the grid and suddenly your plan has a few square meters larger even if you didn't intend it to. So instead We tried to add an already existing grid to the canvas and make it so that the lines drawn would snap onto the grid (see figure 20).

This still led to some problems of accuracy. It proved quite hard to draw a simple square like this as when someone would be a little too far to the left or right from where they started the line would snap to the next cell making it harder to draw a straight line.

And since we are working with a grid of 1 by 1 anyway in the design we ended up with a canvas that would like the others to create an existing grid on canvas but instead people would click inside the cell on the canvas and the corner points of that cell as well as the borders would be created.

Grid generation on canvas

```
# Add a background grid to the canvas
for i in range(0, 400, 40):
    canvas.create_line(i, 0, i, 400, fill="lightgray")
    canvas.create_line(0, i, 400, i, fill="lightgray")
```

Drawing a line that snaps to grid

```
# Initialize variables for drawing
coords = []
lines = []

# Draw lines and store corner points
def draw(event):
    global coords, lines
    x = (event.x // 40) * 40
    y = (event.y // 40) * 40

    if len(coords) > 0:
        last_x, last_y = coords[-1]
        if abs(last_x - x) > abs(last_y - y):
            y = last_y
        else:
            x = last_x

        line_id = canvas.create_line(coords[-1][0], coords[-1][1], x, y, fill="black")
        lines.append(line_id)

        coords.append((x,y))
    return coords
```

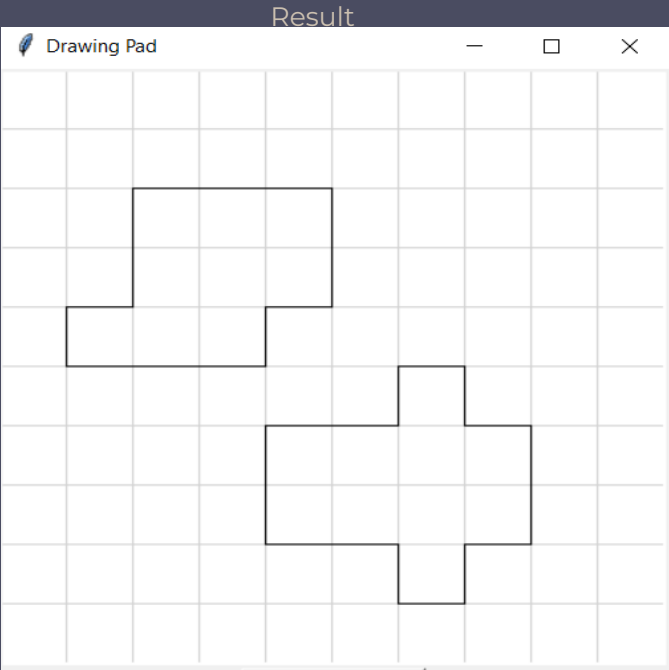


Figure 18, 19 and 20:
Example a different canvas (Drawingtyout.py)
Source: Own code, see <https://github.com/ArtiApple66/Earthquake-Shelter> (github repository)

Changed line to square drawing on canvas

```
def draw(event):
    global coords
    x = (event.x // 40) * 40
    y = (event.y // 40) * 40

    points_to_add = [(x, y), (x, y+40), (x+40, y), (x+40, y+40)]

    for point in points_to_add:
        if point not in coords:
            coords.append(point)
            canvas.create_oval(point[0]-2, point[1]-2, point[0]+2, point[1]+2, fill="blue")

    canvas.create_rectangle(x, y, x+40, y+40, outline="black")
    coords.append((x, y))

    # Store the center of the rectangle
    center = center_of_rectangle(x, y)
    if center not in center_coords:
        center_coords.append(center)

    canvas.bind('<Button-1>', draw)
    canvas.pack()
```

Result

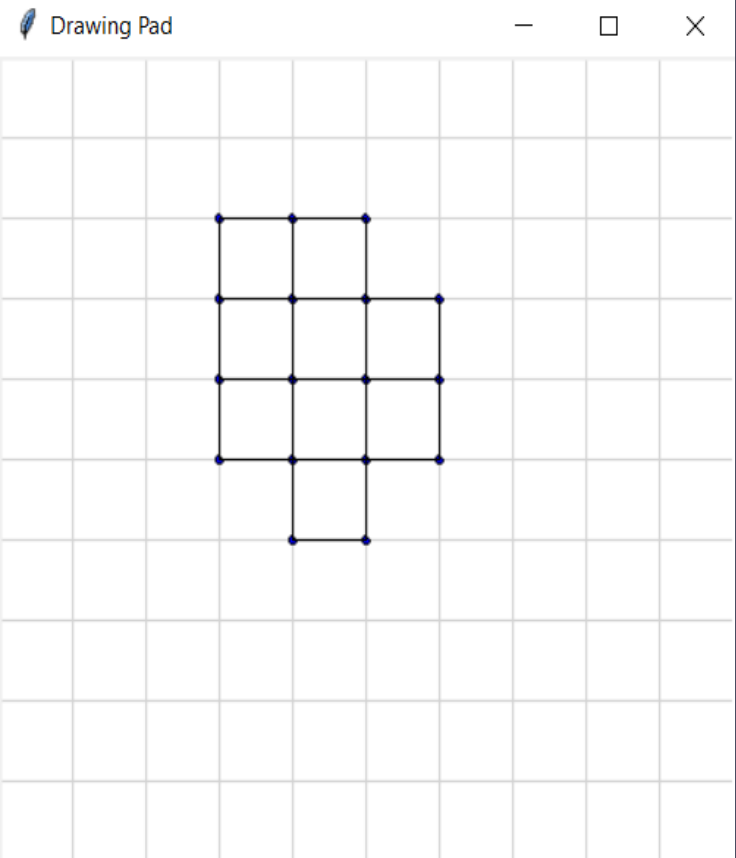


Figure 21, 22 and 23:
Example a different canvas (Drawingtyout.py)
Source: Own code, see <https://github.com/ArtiApple66/Earthquake-Shelter> (github repository)

5.3 The design of the interface

5.3.2 The interface design

Now that we had a way of going from a drawing to a grid, we wanted to create an interface around it for people to hand in their information as well, and send everything to the right place and person.

Our first take was to create a website for the interface. We used Wix to quickly create a template for the website (see figure 25 and visit the site via <https://amirrghadiril.wixsite.com/tsugite-as-shelter>) and wanted to integrate the python script canvas we created with tkinter on the page.

However the problem was that we can't directly interact with the GUI events like we do in a standalone Tkinter application on a webpage. An solution would be to implement the drawing mechanism using JavaScript and HTML5 canvas in the frontend. And then send the drawing data to your Flask backend using AJAX or similar techniques. Meaning we would have had to start over and learn yet another scripting language within a very small time period. So we decided for this project it was easier to continue in the tkinter environment and create an interface there.

As for the inputs we would need from the users from this page would be the following:

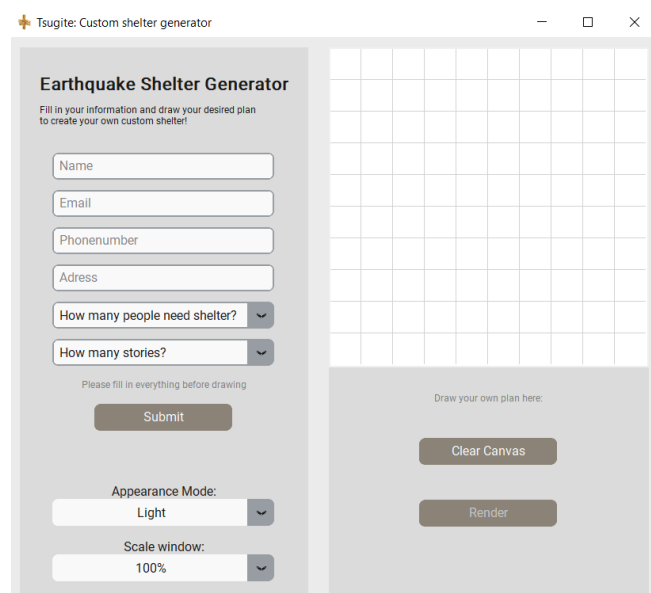
- ◇ String input: Name
- ◇ String input: Email (name@gmail.com)
- ◇ Integer input: phone number (06 1234567)
- ◇ Longitude latitude/ string input: location (street, number, town, land → address)
- ◇ Dropdown menu with integers from 1 to 10: Amount of people needing shelter
- ◇ Dropdown menu with integers from 1 to 3: Amount of stories

This information would once filled in correctly be saved and 'unlock' the canvas. After someone is done drawing with the push of a button it generates the structure, depicts what parts are necessary to build it and sends over the necessary files so the building block can be produced and shipped.

And of course there should also be limitations:

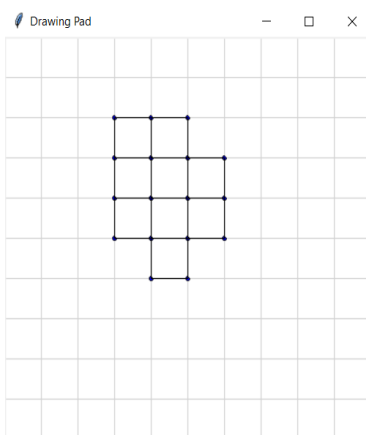
- ◇ The UNHCR states that a shelter should at least be 5.5 square meters. Since we are working with a 1x1 grid we want to give a warning before finally generating the structure if the plan is smaller than 6 square meters per person.
- ◇ The information should be valid and fully filled in before drawing.
- ◇ People should be unable to create weird 'inhabital' spaces.

Figure 24:
Example of the interface
with tkinter
Source: Our code see <https://github.com/ArtiApple66/Earthquake-Shelter> (github repository)
Used a template green.Json from the customtkinter library (extention of tkinter) TomSchimansky. (n.d.). GitHub - TomSchimansky/CustomTkinter: A modern and customizable python UI-library based on Tkinter. GitHub. <https://github.com/TomSchimansky/CustomTkinter>





Draw a plan



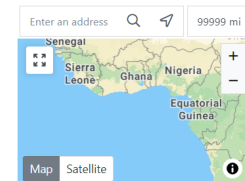
First Name

Last Name

Email *

Address

Address



Select an option

- ☐ No floor will be added in the future to shelter
- ☐ One floor will be added in the future to shelter
- ☐ Two floor floor will be added in the future to shelter

How many people will reside the shelter?

Enter a number

Type your message here...

☐ I accept terms & conditions

Submit

Figure 25: Example interface in HTML

Source: Our own design, visit it on <https://amirrghadiril.wixsite.com/tsugite-as-shelter>

5.3.3 What happens behind the scenes?

For the initial approach we first started out using functional programming for creating the interface. In functional programming, computation is viewed as a series of mathematical functions. These functions take input, perform a specific task without altering the original data (avoiding mutations and state changes), and return a result. However since we were working with complex and hierarchical structures, and wanted to model something that would eventually mirror the real world we opted to eventually step over to object oriented programming (OOP) where as the name implies the code is organized into objects, each of which represents an instance of a class.

So what did we do to create the interface we have now?

1. Used libraries:

- ♦ **Tkinter:** a standard GUI toolkit used to create the set up of the interface as well as the canvas to draw on
- ♦ **Customtkinter:** This is an expansion on the tkinter library to customize the interface
 - > There are 3 preloaded themes, blue, dark-blue and green (of which there also is a dark version), as well as multiple fonts.
 - > In this case we used the green.json file and adapted the colors to grey and beige tints.
- ♦ **Re:** Regular expression operations, for pattern matching
- ♦ **Csv:** For reading and writing Csv files
- ♦ **Numpy:** For creating arrays, sorting, and creating unique values

2. Defining the App Class:

- ♦ The main application class App is defined, inheriting from customtkinter.CTk, which customizes the basic functionality of tkinter.
- ♦ The `__init__` method initializes the GUI elements, including the main window, sidebar, labels, entry fields, buttons, and a canvas for drawing.

3. Handling User Input:

- ♦ Various entry fields (entry_name, entry_email, etc.) are created to capture user information such as name, email, phone number, address, number of people, and number of stories.
- ♦ The `save_personal_info` in combination with the `validate_x` functions validate and save this information to a CSV file (data.csv). It also displays an error message if the input is invalid. The errors are displayed on the interface for various conditions, guiding the user on how to correct their input.

4. Drawing on Canvas:

- ♦ The application allows users to draw rectangles on a canvas using the `draw` method. The drawn corner points of these squares are stored, and the center coordinates of the rectangles are calculated and saved in a CSV file (points.csv) when the “render” button is hit.
- ♦ A button to “Clear Canvas” is provided to clear the canvas as well as the CSV file.
- ♦ The `generate_grid` method creates a guideline grid for the user on the canvas. This will even when clearing the canvas stay on screen.

6. Changing Appearance:

- ♦ The appearance mode (Light, Dark, System) and the scaling of the window can be changed using the respective dropdown menus.

7. Main Execution:

- ♦ The script checks if it's the main module (if `__name__ == "__main__":`) and creates an instance of the App class, initiating the GUI main loop with `app.mainloop()`.

Libraries

```
import tkinter as tk
import customtkinter
import re
import csv
import numpy as np
```

Theme

```
customtkinter.set_appearance_mode("System") # Modes: "System" (standard), "Dark", "Light"
customtkinter.set_default_color_theme("green") # Themes: "blue" (standard), "green", "dark-blue"
```

Class

```
class App(customtkinter.CTk):
    def __init__(self):
        super().__init__()
```

Create layout frames

```
# configure grid layout (3x14)
self.sidebar_frame = customtkinter.CTkFrame(self, width=240, corner_radius=0)
self.sidebar_frame.grid(row=0, column=0, padx=10, pady=10, rowspan=16, columnspan=1, sticky="nsew")
self.sidebar_frame.grid_rowconfigure((0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16), weight=1)
```

The customtkinter interface can divide the screen up in a grid as seen in the table to the side here. So what we wanted is one column for the information of the user, one for the drawing and eventually in the future one where a render would be shown of the generated shelter, and people would be able to download the instructions from.

Columns			
Rows	(0,0)	(1,0)	(2,0)
	(0,1)	(1,1)	(2,1)
	(0,2)	(1,2)	(2,2)
	(0,3)	(1,3)	(2,3)
(Column, row)			

Create inputs and error label

```
# create entries for peoples information
self.entry_name = customtkinter.CTkEntry(self.sidebar_frame, width=225, placeholder_text="Name")
self.entry_name.grid(row=2, column=0, columnspan=1, padx=20, pady=(10, 5))

self.entry_email = customtkinter.CTkEntry(self.sidebar_frame, width=225, placeholder_text="Email")
self.entry_email.grid(row=3, column=0, columnspan=1, padx=20, pady=5)

self.entry_phonenumber = customtkinter.CTkEntry(self.sidebar_frame, width=225, placeholder_text="Phonenumber")
self.entry_phonenumber.grid(row=4, column=0, columnspan=1, padx=20, pady=5)

self.entry_adress = customtkinter.CTkEntry(self.sidebar_frame, width=225, placeholder_text="Adress")
self.entry_adress.grid(row=5, column=0, columnspan=1, padx=20, pady=5)

#amount of people
self.amount_of_people = customtkinter.CTkComboBox(self.sidebar_frame, width=225, values=["How many people need shelter?"] + [str(i) for i in range(1, 11)])
self.amount_of_people.grid(row=6, column=0, padx=20, pady=5)

#amount of people
self.stories = customtkinter.CTkComboBox(self.sidebar_frame, width=225, values=["How many stories?"] + [str(i) for i in range(1, 3)])
self.stories.grid(row=7, column=0, padx=20, pady=5)

#create an error label when someone fills in something incorrectly
self.error_label = customtkinter.CTkLabel(self.sidebar_frame, text="Please fill in everything before drawing", font=customtkinter.CTkFont(size=12))
self.error_label.grid(row=8, column=0, columnspan=1, padx=20, pady=0)
```

Figure 26 to 30: Snippits of the code and a Tkinter grid geometry

Source: Own code, see <https://github.com/ArtiApple66/Earthquake-Shelter> (github repository) Python Tutorial. (2022, September 9). Tkinter Grid Geometry Manager.

Create canvas of 400 by 400 which represents a 1 by 1 grid

```
#create canvas for drawing
self.canvas_frame = customtkinter.CTkFrame(self, corner_radius=0)
self.canvas_frame.grid(row=0, column=1, padx=10, pady=10, rowspan=16, columnspan=2, sticky="nsew")

self.canvas = tk.Canvas(self.canvas_frame, width=400, height=400, bg="white")
self.canvas.grid(row=11, column=1, rowspan = 11, columnspan=2, padx=20, pady=0)

#create background grid on canvas
for i in range(0, 400, 40):
    self.canvas.create_line(i, 0, i, 400, fill="lightgray")
    self.canvas.create_line(0, i, 400, i, fill="lightgray")

self.drawing = False
self.coords = []
self.final_points = []
self.center_coords = []

self.canvas.bind('<Button-1>', self.draw)
self.canvas.pack()
```

Create an errorlabel for the canvas, clear button and render button

```
#create an error label when someone fills in something incorrectly
self.error_label2 = customtkinter.CTkLabel(self, text="Draw your own plan here:", font=customtkinter.CTkFont(size=10), text_color="grey86")
self.error_label2.grid(row=11, column=1, columnspan=2, padx=20, pady=0)

#clear canvas
self.clear_button = customtkinter.CTkButton(self, text="Clear Canvas", command=self.clear_canvas, bg_color="grey86")
self.clear_button.grid(row=12, column=1, columnspan=2, padx=10, pady=5)

#send over the point to generate 3D model
self.render = customtkinter.CTkButton(self, text="Render", command=self.save_to_csv, bg_color="grey86")
self.render.grid(row=13, column=1, rowspan = 1, columnspan=2, padx=10, pady=(0, 10))
self.render.configure(state="disabled")
```

Functions for scaling and changing the theme

```
def change_appearance_mode_event(self, new_appearance_mode: str):
    customtkinter.set_appearance_mode(new_appearance_mode)

def change_scaling_event(self, new_scaling: str):
    new_scaling_float = int(new_scaling.replace("%", "")) / 100
    scaled_width = int(1000 * new_scaling_float) # Scale the width of the main window
    scaled_height = int(580 * new_scaling_float) # Scale the height of the main window
    self.geometry(f"{scaled_width}x{scaled_height}") # Set new dimensions for the main window
    customtkinter.set_widget_scaling(new_scaling_float)
```

Calculating center point of the squares on canvas and saving them to CSV if valid

```
def center_of_rectangle(self, x, y):
    return x + 20, y + 20

def save_to_csv(self):
    global square_generated
    num_squares = len(self.center_coords) # Number of squares generated
    total_area = num_squares # Area covered by squares

    if not square_generated:
        self.error_label2.configure(text="Please draw a plan first.", text_color="red")
    elif (int(self.amount_of_people.get()) * 6) > total_area:
        print ((int(self.amount_of_people.get()) * 6), total_area)
        self.error_label2.configure(text="Area is too small for the specified number of people.", text_color="red")
    else:
        with open('points.csv', mode='a', newline='') as file:
            writer = csv.writer(file)
            writer.writerow(["X", "Y"]) # Header
            writer.writerows(self.center_coords)
            print("saved")
        self.error_label2.configure(text="Information successfully saved!", text_color="green")
```

Figure 31 to 36: Snippets of the code and a Tkinter grid geometry Source: Own code, see <https://github.com/Al>

Functions of validation for the inputs

```
def is_valid_name(self, name):
    pattern = r'^[a-zA-Z- ]+$'
    return re.match(pattern, name) is not None

def is_valid_email(self, email):
    # Regular expression to validate email format
    pattern = r'^[a-zA-Z0-9_+-.]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-]+$'
    return re.match(pattern, email) is not None

def is_valid_phonenumber(self, phonenumber):
    # Regular expression to validate phone number format (only digits)
    pattern = r'^\d+$'
    return re.match(pattern, phonenumber) is not None

def is_valid_address(self):
    # Split the address into components
    components = self.entry_address.get().split(',')

    # Check if all required components are present
    if len(components) >= 4:
        street = components[0].strip()
        house_number = components[1].strip()
        town = components[2].strip()
        country = components[3].strip()

        # Check if each component is not empty
        if street and house_number and town and country:
            return True

    return False
```

Displaying the right error label when not valid or save to CSV

```
def save_personal_info(self):
    name = self.entry_name.get()
    email = self.entry_email.get()
    phonenumber = self.entry_phonenumber.get()
    address = self.entry_address.get()
    people = self.amount_of_people.get()
    stories = self.stories.get()

    # Check if everything is provided
    if (
        name
        and email
        and phonenumber
        and address
        and people != "How many people need shelter?"
        and stories != "How many stories?"
        and self.is_valid_name(name)
        and self.is_valid_email(email)
        and self.is_valid_phonenumber(phonenumber)
        and self.is_valid_address()
    ):
        # Open the CSV file in append mode and write the information
        with open('data.csv', mode='a', newline='') as file:
            writer = csv.writer(file)
            writer.writerow(["name", "email", "phonenumber", "address", "people", "stories"])
            writer.writerow([name, email, phonenumber, address, people, stories])
            print(f"Saved {name}, {email}, {phonenumber}, {address}, {people}, {stories} to data.csv")
        # Reset error message if it was previously displayed
        self.error_label.configure(text="Information successfully saved!", text_color="green")
        self.render.configure(state="normal")
    else:
        if not self.is_valid_name(name):
            self.error_label.configure(text="Please give a valid name", text_color="red")
        elif not self.is_valid_email(email):
            self.error_label.configure(text="Please give a valid email adress", text_color="red")
        elif not self.is_valid_phonenumber(phonenumber):
            self.error_label.configure(text="Please give a valid phonenumber (061234567)", text_color="red")
        elif not self.is_valid_address():
            self.error_label.configure(text="Please give adress like: street, housenumber, town, country", text_color="red")
        else:
            self.error_label.configure(text="Please fill in all the boxes correctly", text_color="red")
```


5.4 Model generation

5.4.1 How to communicate with rhino, grasshopper

We choose to do the visualisation and generation of the structure in rhino grasshopper since this would easily enable us to automate multiple events, like the structural analysis with the Karamba plugin. One of the challenges however was to get the information from the interface to grasshopper. Our first approach was to load them through a Hops component since this would update the list live (see `drawinghops.py`). So when you'd make a change you would instantly (or after a loading period) see results. This is done by creating a flask application and load the points through there. This however proved quite hard to get right. The component was only able to send over rhino 'points', and would also break if the points would simply be saved as `[(x,y,z)]` instead of something rhino could read (`<rhino3dm._rhino3dm.Point3d object at 0x000002A30E020CB0>`). As well as the fact that it would sometimes work flawlessly and other times with the same piece of code not work at all. So we thought it to be unpredictable so we stepped over to a different approach.

So instead after someone fills in their information and sends over the point list to the CSV file we choose to generate the structure or actual model from these points. In the provided code, there are two main functions that work with CSV data and 3D point manipulation using a scripting language for Rhino, which is a 3D modeling software.

Looking at the code on the right, the first function, `read_csv(filepath)`, is designed to read coordinates from a CSV file and create points in Rhino. It begins by opening the specified CSV file and uses Python's `csv.reader` to parse it. The function skips the header row with `next(reader)` and then iterates over each row of data. If a row contains less than two elements, it is ignored. For the remaining rows, it attempts to convert the first two values into floating-point numbers and scales them down by a factor of 40. If the conversion is successful, a 3D point is created at the scaled x, y coordinates with a z-value of 0 using Rhino's scripting syntax `rs.AddPoint`. If a `ValueError` occurs (which would happen if the row contains non-numeric data), an error message is printed. The function returns a list of points that were created.

The second function, `create_squares(centers)`, takes a list of points that represent centers and creates squares with a side length of 1 unit centered on these points. For each point, it calculates the coordinates of the square's corners by moving half a unit in the positive and negative directions along the x and y axes. A polyline is then created that connects these corners in a square shape, and each polyline is added to a list of squares.

Finally, the code sets the variable `csv_path` to the path of a CSV file and calls `read_csv` with this path to create points. It then calls `create_squares` with the list of points returned by `read_csv`, resulting in a list of squares centered on each point imported from the CSV file. This script essentially reads point data from a file and visualizes each point as the center of a square within the Rhino 3D environment.

5.2.1 Getting the right plan

One of the more challenging aspects of the project was in this part of the process. The reason we choose to opt for generating squares based on their centers, rather than their vertices, stems from the challenge of determining which points should be connected to form a square. Initially, when attempting to create squares by directly linking vertex points, the code struggled to accurately identify adjacent points that constitute a square. This is analogous to the limitations of a Convex Hull approach, which tends to encompass the largest possible area, thus not suitable for creating multiple distinct squares. By receiving coordinates for the centers, we can precisely construct identical squares as intended by the user. This method ensures that each square is formed accurately by calculating and defining the positions of its vertices relative to its center, guaranteeing the creation of uniform squares with the correct orientation and size.

Creating the Flask application

```
# register hops app as middleware
app = Flask(__name__)
hops = hs.Hops(app)
```

Adding the path and output of the app (a definition should follow this)

```
@hops.component(
    "/grid",
    name="pointgridfinal",
    nickname="gg",
    description="generating grid based on drawing",
    outputs=[
        hs.HopsPoint("pointgridfinal", "point_grid", "Final Points", hs.HopsParamAccess.LIST),
    ]
)
```

Figure 37 and 38: Snippets of the code for using a hops component

Source: Own code, see <https://github.com/ArtiApple66/Earthquake-Shelter> (github repository)

```
import csv
import rhinoscriptsyntax as rs

def read_csv(filepath):
    points = []
    with open(filepath, 'r') as file:
        reader = csv.reader(file)
        next(reader) # Skip the header
        for row in reader:
            if len(row) < 2: # Check if there are less than 2 values in the row
                continue # Skip this row
            try:
                x, y = map(float, row)
                x /= 40
                y /= 40
                points.append(rs.AddPoint(x, y, 0))
            except ValueError: # Handle possible value errors (like non-numeric d
                print("Error processing row:", row)
    return points

def create_squares(centers):
    half_side = 0.5 # As the total side is 1
    squares = []
    for pt in centers:
        x, y, z = rs.PointCoordinates(pt)
        corner1 = rs.AddPoint(x - half_side, y - half_side, z)
        corner2 = rs.AddPoint(x + half_side, y - half_side, z)
        corner3 = rs.AddPoint(x + half_side, y + half_side, z)
        corner4 = rs.AddPoint(x - half_side, y + half_side, z)
        square = rs.AddPolyline([corner1, corner2, corner3, corner4, corner1])
        squares.append(square)
    return squares

# Using the path you provided
csv_path = r"C:\Users\Asus\Desktop\coordinates_csv\exported_points.csv"
points = read_csv(csv_path)

# Create squares with centers as the imported points
squares = create_squares(points)
```

Figure 39: The python component in grasshopper reading the CSV file

Source: Own code (parametrized design (3).gh)

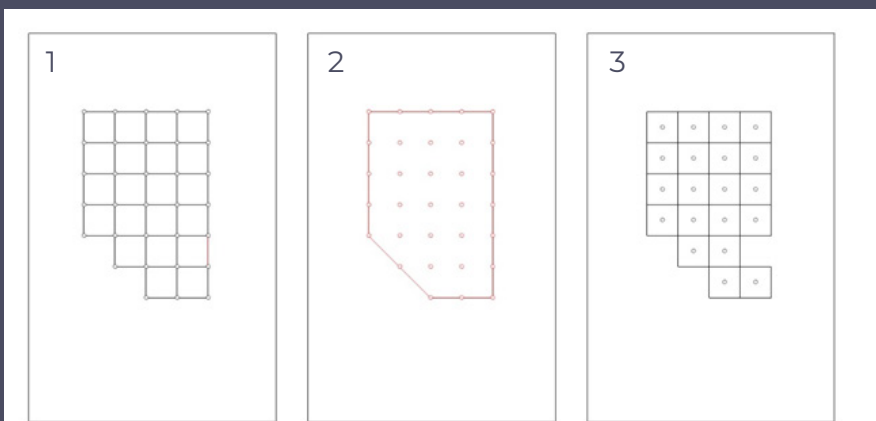


Figure 40:

1. Shows the inaccuracies when attempting to create square borders using vertex coordinates, leading to incorrect linkages.

2. The failure of the Convex Hull method, which results in an overly broad area that doesn't match the intended multiple square shapes.

3. Displays the successful creation of the desired squares, achieved by calculating from the central points, ensuring precise and uniform squares as originally designed by the user.

5.4.2 Creating a grid

So now we need to create a grid with those points to get to our structure. The code on the left demonstrates an approach of creating structured grid systems within a 3D space, specifically for the first level of a given structure. The process involves dividing curves, generating grids, and sorting points to establish a coherent spatial organization. Here's an explanation of the various sections and their functions:

1. Dividing Curves by Length:

- ◇ Function `divide_curve_by_length`: This function divides a given curve into segments of specified length, returning a list of division points.
- ◇ Application: It is used to divide the curve `Ln_1[0]` into segments, creating a series of points (`Pnt_L1_T1`).

2. Creating Grid Inside a Curve:

- ◇ Function `create_grid_inside_curve`: This generates a grid of points within the bounding box of a specified curve, only including points that fall inside the curve.
- ◇ Application: Utilized to create a grid (`grid_points_temp_1`) inside the curve `curve_to_divide_1`.

3. Filtering Grid Points:

- ◇ Tolerance Definition: A small tolerance is set for comparing points.
- ◇ Filtering Process: Points in `grid_points_temp_1` that are sufficiently far from the points in `Pnt_L1_T1` are collected into `Pnt_L1_T2`.

4. Sorting and Selecting Points:

- ◇ Sorting: `Pnt_L1_T2` points are sorted by their Y and then X coordinates.
- ◇ Grouping and Selection: Points are grouped by their Y coordinate, and every other point from each group is selected to form `Pnt_L1_T3`.

5. Point Manipulation for Multiple Levels:

- ◇ Level 3 Points: Points from `Pnt_L1_T1`, `Pnt_L1_T2`, `Pnt_L1_T3` are copied and moved to create corresponding points for the third level (`Pnt_L3_T1`, `Pnt_L3_T2`, `Pnt_L3_T3`).
- ◇ Level 2 Points: Similarly, points for the second level (`Pnt_L2_T1`) are created from `Pnt_L1_T1`.

6. Generating Grid Curves:

- ◇ Function `create_curves_from_points`: Automates the process of creating curves from a list of points.
- ◇ Grid Creation for Levels: Uses the generated points from each level to create a network of curves (`curves_L1`, `curves_L2`, `curves_L3`).

7. Connecting Levels:

- ◇ Connecting Level 1 and 2: Creates curves (`curves_L12`) connecting points between levels 1 and 2.
- ◇ Connecting Level 2 and 3: Similar process for levels 2 and 3, generating `curves_L23`.

8. Combining Curves:

- ◇ Combination of Curves: All the curves created (`Crvsxy` and `Crvsz`) are combined to form the structural grid for the shelter.

9. Identifying Beams and Columns:

- ◇ Curve Orientation Analysis: Distinguishes between x-oriented and y-oriented curves based on their midpoints and orientation, categorizing them as either beams (`curves_x`, `curves_y`) or columns.
- ◇ Midpoints Calculation: Determines the midpoints of these curves for structural analysis or further manipulation.

10. Midpoints for Vertical Elements

- ◇ Midpoint Calculation for Vertical Elements: Midpoints of curves connecting different levels (`Crvsz`) are calculated, likely representing column locations or key structural points.

Conclusion

This code forms the foundation of a complex architectural model, establishing a multi-level grid system that can be utilized for various design and analysis purposes. It showcases a sophisticated use of geometric computation, crucial in modern architectural design and structural engineering.

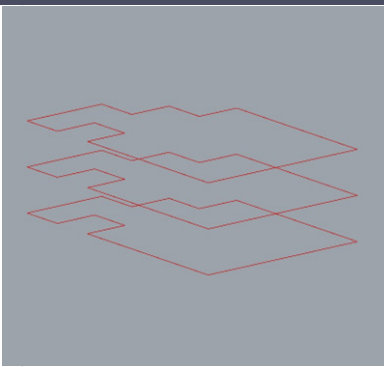
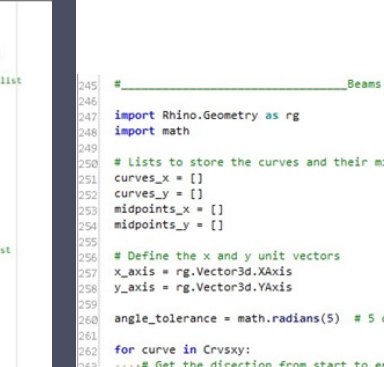
<pre> 9 10 def divide_curve_by_length(curve, length=1.0): 11 t_values = rg.Curve.DivideByLength(curve, length, True) 12 if not t_values: 13 return [] 14 division_points = [curve.PointAt(t) for t in t_values] 15 return division_points 16 17 # Pnt_L1_T1 18 curve_to_divide_1 = rs.coercecurve(ln_1[0]) 19 Pnt_L1_T1 = divide_curve_by_length(curve_to_divide_1, 1.0) 20 21 22 23 24 25 26 27 28 29 import Rhino.Geometry as rg 30 31 def create_grid_inside_curve(curve, grid_size): 32 """Get the bounding box of the curve 33 bboxes = curve.GetBoundingBox(True) 34 """ 35 """Define the start and end points for the x and y axes 36 x_start, x_end = bboxes.Min.X, bboxes.Max.X 37 y_start, y_end = bboxes.Min.Y, bboxes.Max.Y 38 """ 39 """Generate grid points 40 grid_points_temp_1 = [] 41 """ 42 x = x_start 43 while x <= x_end: 44 y = y_start 45 while y <= y_end: 46 pt = rg.Point3d(x, y, 0) 47 if rg.Curve.Contains(curve, pt) != rg.PointContainment.Outside: # Check if the point is inside the curve 48 grid_points_temp_1.append(pt) 49 y = grid_size 50 x += grid_size 51 x = grid_size 52 return grid_points_temp_1 53 54 # Inputs: 55 # ln_1 - The input curve 56 # grid_size - The size of each grid cell. Set this to 1.0 for a 1x1 grid. 57 58 grid_size = 1.0 59 grid_points_temp_1 = create_grid_inside_curve(curve_to_divide_1, grid_size) 60 61 tolerance = 1e-6 # Set a small tolerance for point comparison 62 63 Pnt_L1_T2 = [pt for pt in grid_points_temp_1 if all(pt.DistanceTo(pt2) > tolerance for pt2 in Pnt_L1_T1)] 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 </pre>		<pre> 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 </pre>
<pre> 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 </pre>		<pre> 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 </pre>

Figure 41 to 44: The pieces of the code that generates the structure in rhino, grasshopper gh-python Source: Own code and screen shot (see grasshopper parametrized design (3).gh for higher resolution example).

5.4.3 Generating the joints and elements of the structure

This code is an essential part of designing a complex geometric joint connection for a shelter structure. It intricately combines mathematical calculations and geometric transformations to create a series of interconnected cuboids, which form the basis of the joint.

1. Initial Calculations and Point Definitions

- ◇ Linear Measurements: The code begins with calculating linear measurements like `L_Ta`, which are used to define the dimensions of various parts of the geometry.
- ◇ Point Creation: Crucial points like `o_a_1`, `o_a_2`, `o_a_3` are defined using coordinates. These points serve as references for positioning the geometric elements in space.

2. Plane and Cuboid Generation

- ◇ Planes: Planes like `plane_a1`, `plane_a2`, and `plane_a3` are defined. These are instrumental in establishing the orientation of the cuboids in 3D space.
- ◇ Cuboids Creation: Cuboids `a1`, `a2`, `a3` are then generated using these planes and intervals. These cuboids represent the primary components of the joint.

3. Mirroring and Duplication

- ◇ Mirroring: To achieve symmetry, a mirroring transformation is applied to the initial cuboids. This involves defining a mirror plane and then using it to create mirrored versions of the original cuboids (`a4`, `a5`, `a6`).

4. Additional Geometric Components

- ◇ Secondary Points and Boxes: The code continues to define additional points (`O_B1`, `O_B2`, etc.) and uses them to create more geometric components (`boxes_small`, `b9`, `b10`). These elements add complexity and functionality to the joint.
- ◇ Box Creation Function: A function `create_box` is defined to streamline the creation of boxes with varying dimensions.
- ◇ Further Expansion with Complex Geometry
- ◇ Tertiary Component Creation: The script further expands with the introduction of tertiary components (`boxes_c`, `c9`, `c10`). These are created following a similar pattern but with different dimensions and positions.

5. Assembling the Joint

- ◇ Combining Components: All individual components (`A`, `B`, `C`) are combined into a single joint structure, represented by the variable `J`.
- ◇ Rotation Transformation: The joint structure undergoes a rotation transformation. This step is crucial for aligning the joint according to the shelter structure's requirements.
- ◇ Output of Rotated Joint: The rotated joint components are stored in the list `JR`, ready for integration into the shelter structure.

This code adeptly demonstrates the use of geometric transformations, point-based construction, and spatial reasoning in computational design. The resulting geometry is not just a collection of shapes but a carefully orchestrated assembly of components, poised to function as a joint in a shelter structure. The meticulous attention to spatial relationships and dimensions ensures that the joint will seamlessly integrate with other structural elements, contributing to the stability and aesthetic of the overall design.

In this phase of the design process, a parametric design platform was utilized to synthesize complex geometries for the structural integrity of a shelter into a series of joint types. An intricate node-based system was constructed to methodically generate and manipulate geometric data. This allowed for the precise orchestration of operations such as scaling, rotating, and combining basic shapes, resulting in nine specialized joint types. Each type was engineered to meet specific functional requirements at various locations within the shelter structure. Leveraging the robust capabilities of the visual programming environment ensured that each joint was not only structurally sound but also optimized for its intended application, showcasing the potential of computational methods in bespoke architectural fabrication.


```

11 # Calculations and points
12 l_Ta = lA - (2 * T_Ta)
13 O = rg.Point3d(0, 0, 0)
14 o_a_1 = rg.Point3d((d/2)+(wa/2), 0, 0)
15 o_a_2 = rg.Point3d((d/2)+(wa)+(w_ta/2), 0, 0)
16 o_a_3 = rg.Point3d((d/2)+(wa)+(w_ta/2), 0, 0)
17
18 # Planes and cuboids
19 plane_a1 = rg.Plane(o_a_1, rg.Vector3d.ZAxis)
20 plane_a2 = rg.Plane(o_a_2, rg.Vector3d.ZAxis)
21 plane_a3 = rg.Plane(o_a_3, rg.Vector3d.ZAxis)
22
23 a1 = rg.Box(plane_a1, rg.Interval(-wa/2, wa/2), rg.Interval(-lA/2, lA/2), rg.Interval(-T/2, T/2))
24 a2 = rg.Box(plane_a2, rg.Interval(-w_ta/2, w_ta/2), rg.Interval(-l_Ta/2, l_Ta/2), rg.Interval(-T/2, T/2))
25 a3 = rg.Box(plane_a3, rg.Interval(-wa/2, wa/2), rg.Interval(-lA/2, lA/2), rg.Interval(-T/2, T/2))
26
27 # Outputs
28 a = a1
29 b = a2
30 c = a3
31
32
33
34 # Defining the mirror plane
35 mirror_plane = rg.Plane(0, rg.Vector3d.XAxis) # ZY plane through O
36
37 # Create a mirror transformation
38 mirror_transform = rg.Transform.Mirror(mirror_plane)
39
40 # Mirroring the cuboids by transforming their Brep representations
41 a4 = rg.Box.ToBrep(a1)
42 a4.Transform(mirror_transform)
43
44 a5 = rg.Box.ToBrep(a2)
45 a5.Transform(mirror_transform)
46
47 a6 = rg.Box.ToBrep(a3)
48 a6.Transform(mirror_transform)
49
50 A = [a1, a2, a3, a4, a5, a6]
51
52 # Calculated coordinates
53 bx = ((w_tb) + (wb)) / 2
54 by = (d_B + Tb) / 2
55 bz = ((T / 2) + ((lB - T) / 4))
56
57 # Creating points
58 O_B1 = rg.Point3d(0, (d_B / 2) + (Tb / 2), 0)
59 O_B2 = rg.Point3d(0, -((d_B / 2) + (Tb / 2)), 0)
60 O_B1 = rg.Point3d(bx, by, bz)
61 O_B2 = rg.Point3d(-bx, by, bz)
62 O_B3 = rg.Point3d(bx, by, -bz)
63 O_B4 = rg.Point3d(-bx, by, -bz)
64 O_B5 = rg.Point3d(bx, -by, bz)
65 O_B6 = rg.Point3d(-bx, -by, bz)
66 O_B7 = rg.Point3d(bx, -by, -bz)
67 O_B8 = rg.Point3d(-bx, -by, -bz)

```

```

68 # Output
69 points = [O_B1, O_B2, O_B3, O_B4, O_B5, O_B6, O_B7, O_B8]
70
71 # Assign to the Grasshopper output variable
72 bp = points
73
74 # Additional definition
75 l_Tb = lB - (2 * T_Tb)
76 lB = (lB - T) / 2
77
78 # Function to create a box centered at a point with given dimensions
79 def create_box(center, x, y, z):
80     base_plane = rg.Plane(center, rg.Vector3d.ZAxis)
81     interval_x = rg.Interval(-x / 2, x / 2)
82     interval_y = rg.Interval(-y / 2, y / 2)
83     interval_z = rg.Interval(-z / 2, z / 2)
84     return rg.Box(base_plane, interval_x, interval_y, interval_z)
85
86 # Creating the boxes
87 boxes_small = [
88     create_box(pt, wb, Tb, lB) for pt in [O_B1, O_B2, O_B3, O_B4, O_B5, O_B6, O_B7, O_B8]
89 ]
90
91 b9 = create_box(O_B1, w_tb, Tb, l_Tb)
92 b10 = create_box(O_B2, w_tb, Tb, l_Tb)
93
94 B = boxes_small + [b9, b10]
95
96 # Your provided expressions:
97 l_Tc = lC - (2 * T_Tc) # Correcting T_Tc to TC based on subsequent expressions
98 Tc_T = (w_tb) + (2 * wb)
99 lC = (lC - Tc) / 2
100
101 cx = (Tc_T + lC) / 2
102 cy = (wC + w_tc) / 2
103 cz = (T + Tb) / 2
104
105 # Creating points according to your provided coordinates:
106 O_C1 = rg.Point3d(0, 0, cz)
107 O_C2 = rg.Point3d(0, 0, -cz)
108 O_C1 = rg.Point3d(cx, cy, cz)
109 O_C2 = rg.Point3d(-cx, cy, cz) # Correcting the typo from cyz to cz
110 O_C3 = rg.Point3d(cx, -cy, cz)
111 O_C4 = rg.Point3d(-cx, -cy, cz)
112 O_C5 = rg.Point3d(cx, cy, -cz)
113 O_C6 = rg.Point3d(-cx, cy, -cz)
114 O_C7 = rg.Point3d(cx, -cy, -cz)
115 O_C8 = rg.Point3d(-cx, -cy, -cz)
116
117
118 # For example, if the output variable is called points, you would write:
119 points = [O_C1, O_C2, O_C3, O_C4, O_C5, O_C6, O_C7, O_C8]
120

```

```

121 # Creating boxes centered at previously defined points
122 def create_box(center, x_length, y_width, z_height):
123     base_plane = rg.Plane(center, rg.Vector3d.ZAxis)
124     interval_x = rg.Interval(-x_length/2, x_length/2)
125     interval_y = rg.Interval(-y_width/2, y_width/2)
126     interval_z = rg.Interval(-z_height/2, z_height/2)
127     return rg.Box(base_plane, interval_x, interval_y, interval_z)
128
129 # Cuboids c1 to c8
130 boxes_c = [
131     create_box(pt, lC, wC, Tc) for pt in [O_C1, O_C2, O_C3, O_C4, O_C5, O_C6, O_C7, O_C8]
132 ]
133
134 # Cuboids c9 and c10
135 c9 = create_box(O_C1, l_Tc, w_tc, Tc)
136 c10 = create_box(O_C2, l_Tc, w_tc, Tc)
137
138 # To output these boxes from your GH/Python component, assign them to output variables.
139 # If the output variables are named boxes_c_out and boxes_9_10_out, you would write:
140 boxes_c_out = boxes_c
141 boxes_9_10_out = [c9, c10]
142 C=boxes_c+ boxes_9_10_out
143
144 #Combining all the boxes as one joints
145 J=A+B+C
146
147 # Rotation angle in degrees (90 degrees for y-axis)
148 angle = 90
149
150 # Initialize an empty list to store the rotated boxes
151 JR = []
152
153 # Loop through each box in the list and rotate it
154 for box in J:
155     rotated_box = rs.RotateObject(box, [0, 0, 0], angle, [0, 1, 0])
156     JR.append(rotated_box)
157
158 # Output the list of rotated boxes
159 JR

```

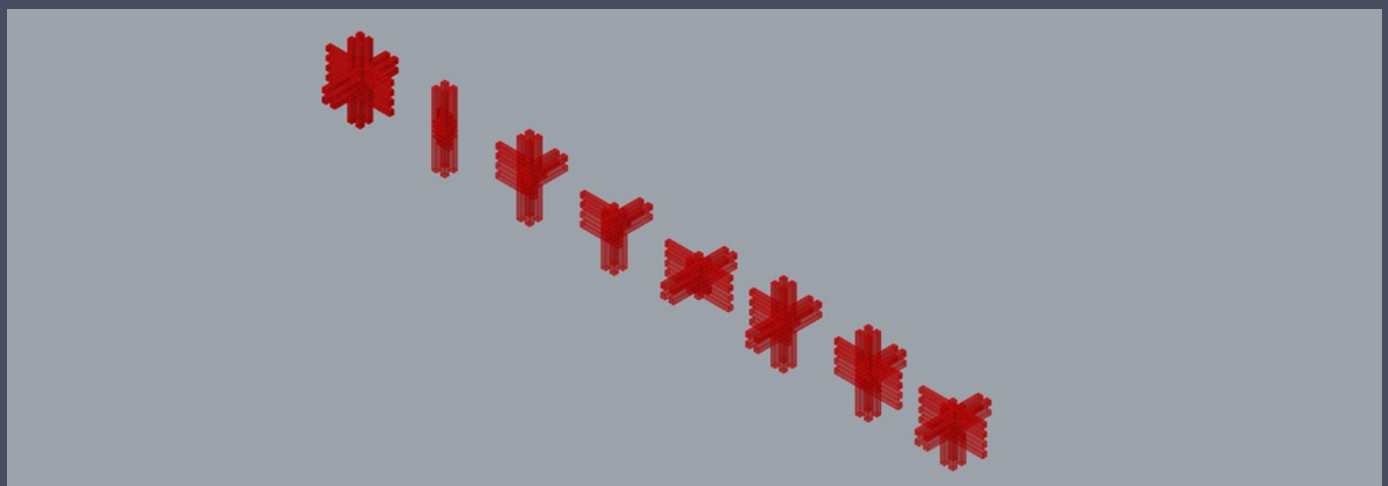
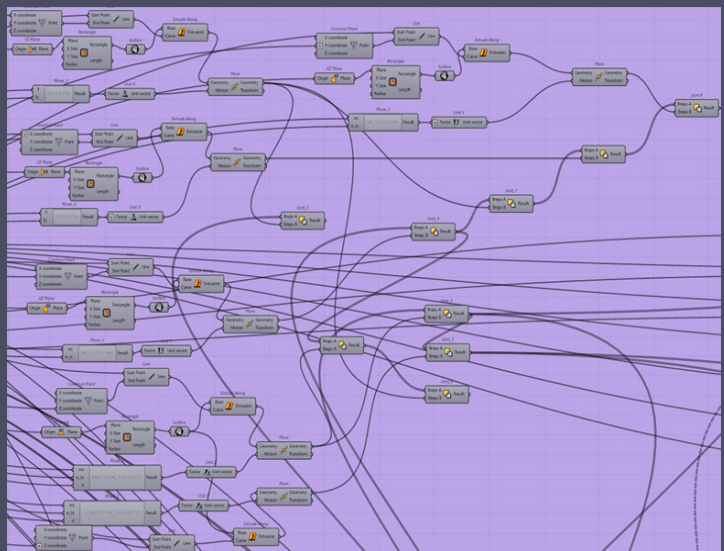


Figure 45 to 49: The pieces of the code that generates the structure in rhino, grasshopper gh-pyhton and results
Source: Own code and screen shots from rhino (parametrized design (3).gh)

5.4.3 Generating beams and coulmsns

In the construction of the shelter, the precise placement of joints along the defined axes was a crucial step. This was accomplished by initially establishing a reference point 'O', along with unit points along the X, Y, and Z axes, denoted as O_x, O_y, and O_z, respectively. For each axis, empty lists Joints_x, Joints_y, and Joints_z were prepared to store the moved joint geometries. The process involved copying the original joint geometry (JRG) and translating it along each axis to the unit points. The translation vectors were computed by subtracting the reference point from each unit point. These vectors directed the movement of the copied geometries to their new positions. As a result, three sets of joints were positioned along the X, Y, and Z axes, populating the lists Joints_x, Joints_y, and Joints_z.

Subsequently, the process shifted to the creation of beams and columns. The midpoint for each directional beam was calculated as the average of the coordinates of the reference point and the corresponding unit point. This established the base points ob_x, ob_y, and ob_z for the beams along the X, Y, and Z axes.

Beams along the X and Y axes were then defined by their thickness, width, and length, with these dimensions encapsulated within a box geometry. For the X-directional beam (Beam_x), dimensions were derived from the beam depth d_B, the thickness Tb, and the width parameters w_tb and wb. Similarly, for the Y-directional beam (Beam_y), dimensions were constructed around the parameters wa, d, w_ta, and the thickness T.

Finally, the column along the Z-axis (Column_z) was created with its dimensions determined by the thickness T, the clearance Tc, the width wc, and the connecting width w_tc. For each beam and column, a plane was established at their respective base points, and intervals were calculated based on their dimensions to form the final box geometries representing the structural elements. This meticulous process ensured that each beam and column would not only serve its structural purpose but also align perfectly with the pre-determined grid system of the shelter.

The supporting pillars, constructed using Grasshopper and GH Python, will be integrated to enhance stability and counteract bending in larger spans. Similarly, the supporting braces, created using Grasshopper and GH Python, will be incorporated to bolster stability against lateral loads. Initially, we attempted to utilize GH Python for positioning each element, but encountered challenges with transferring data between GH Python components, leading to numerous errors. Consequently, we opted for Grasshopper's two-vector move component to accurately place each crafted element in its designated location. Nevertheless, all the points for locating the elements were generated using GH Python.

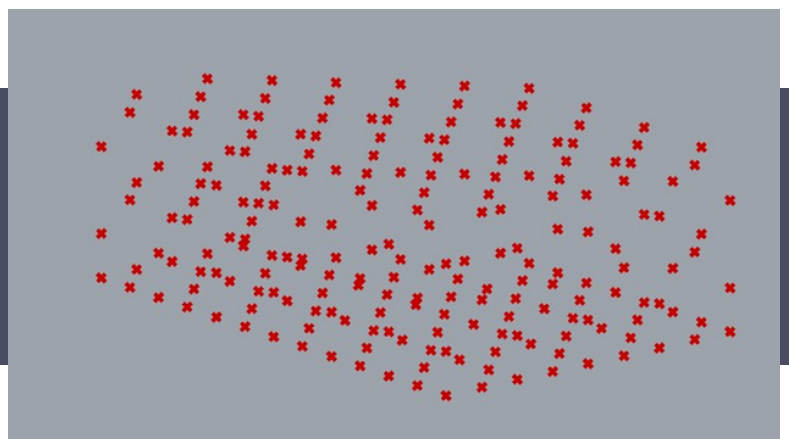
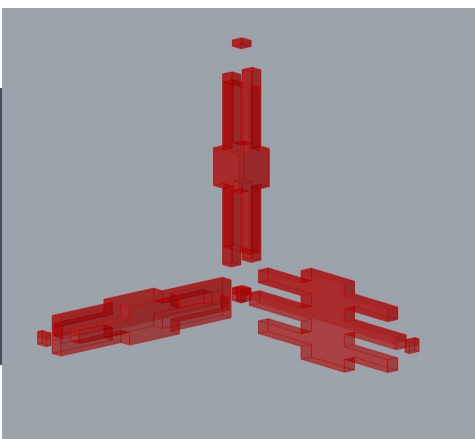


Figure 50 and 51: The results of the code
Source: Own code and screen shots from rhino

```

3 #----- moving joints-----
4 point_0 = rs.PointCoordinates(0)
5 O_x=rg.Point3d(1, 0, 0)
6 O_y=rg.Point3d(0, 1, 0)
7 O_z=rg.Point3d(0, 0, 1)
8
9 Joints_x=[]
10 Joints_y=[]
11 Joints_z=[]
12
13 copied_geometry = rs.CopyObject(JRG) # Create a copy of the input geometry
14
15
16 for x in JRG:
17     copied_geometry = rs.CopyObject(x) # Create a copy of the input geometry
18     translation_vector = O_x - point_0 # Calculate the translation vector from point "0" to the destination point
19     rs.MoveObject(copied_geometry, translation_vector) # Move the copied geometry to the destination point
20     Joints_x.append(copied_geometry)
21
22 for y in JRG:
23     copied_geometry = rs.CopyObject(y) # Create a copy of the input geometry
24     translation_vector = O_y - point_0 # Calculate the translation vector from point "0" to the destination point
25     rs.MoveObject(copied_geometry, translation_vector) # Move the copied geometry to the destination point
26     Joints_y.append(copied_geometry)
27
28 for z in JRG:
29     copied_geometry = rs.CopyObject(z) # Create a copy of the input geometry
30     translation_vector = O_z - point_0 # Calculate the translation vector from point "0" to the destination point
31     rs.MoveObject(copied_geometry, translation_vector) # Move the copied geometry to the destination point
32     Joints_z.append(copied_geometry)
33
34 #----- create beams_columns-----
35
36 # Calculate the middle point x
37 ob_x = rg.Point3d((point_0.X + O_x.X) * 0.5,
38                  (point_0.Y + O_x.Y) * 0.5,
39                  (point_0.Z + O_x.Z) * 0.5)
40
41 # Calculate the middle point y
42 ob_y = rg.Point3d((point_0.X + O_y.X) * 0.5,
43                  (point_0.Y + O_y.Y) * 0.5,
44                  (point_0.Z + O_y.Z) * 0.5)

```

```

52 #Create the beam in x direction
53 B_x_tck= (d*B)+ 2*(Tb)
54 B_x_wdt= ((w_ttb) + 2*(wb))
55 B_x_lng= 1
56
57 B_x_tck = d*B + 2 * Tb
58 B_x_wdt = w_ttb + 2 * wb
59 B_x_lng = 1
60
61 plane = rg.Plane(ob_x, rg.Vector3d.ZAxis)
62
63 # Calculate intervals based on the given dimensions
64 x_interval = rg.Interval(-B_x_lng/2, B_x_lng/2)
65 y_interval = rg.Interval(-B_x_wdt/2, B_x_wdt/2)
66 z_interval = rg.Interval(-B_x_tck/2, B_x_tck/2)
67
68 # Create the beam x
69 Beam_x = rg.Box(plane, x_interval, y_interval, z_interval)
70
71 B_y_tck = (4*wa) + d + (2*w_ta)
72 B_y_wdt = T
73 B_y_lng = 1
74
75 # Define the base plane centered at ob_y
76 plane = rg.Plane(ob_y, rg.Vector3d.ZAxis)
77
78 # Calculate intervals based on the given dimensions
79 x_interval = rg.Interval(-B_y_wdt/2, B_y_wdt/2)
80 y_interval = rg.Interval(-B_y_lng/2, B_y_lng/2)
81 z_interval = rg.Interval(-B_y_tck/2, B_y_tck/2)
82
83 # Create the beam y
84 Beam_y = rg.Box(plane, x_interval, y_interval, z_interval)
85
86 # Create the beamz
87 B_z_tck = T + (2*Tc)
88 B_z_wdt = (2*wc) + w_tc
89 B_z_lng = 1
90
91 # Define the base plane centered at ob_z
92 plane = rg.Plane(ob_z, rg.Vector3d.ZAxis)
93
94 # Calculate intervals based on the given dimensions
95 x_interval = rg.Interval(-B_z_tck/2, B_z_tck/2)
96 y_interval = rg.Interval(-B_z_wdt/2, B_z_wdt/2)
97 z_interval = rg.Interval(-B_z_lng/2, B_z_lng/2)
98
99 # Create the box
100 Column_z = rg.Box(plane, x_interval, y_interval, z_interval)

```

Figure 52 and 53: The pieces of the code that generates the structure in rhino, grasshopper gh-pyhton
Source: Own code and screen shots from rhino (parametrized design (3).gh)

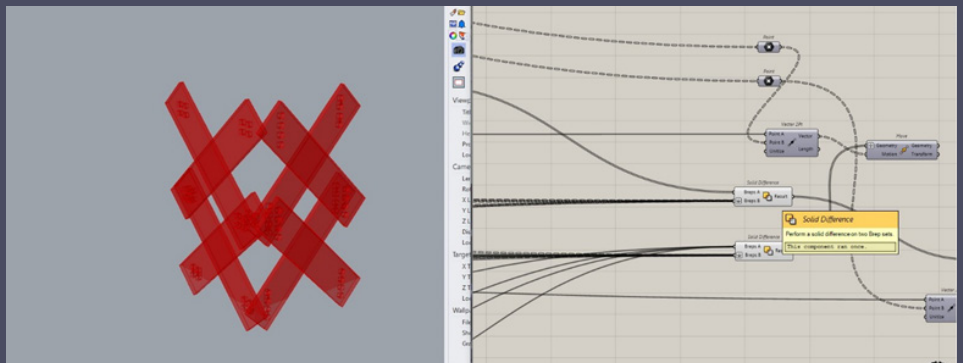
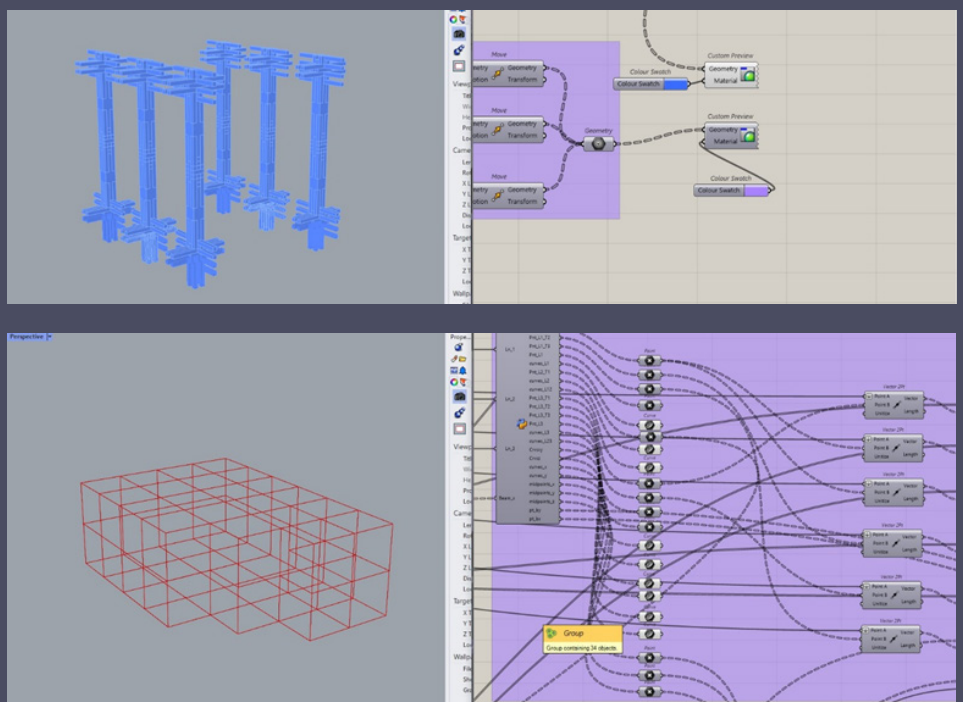


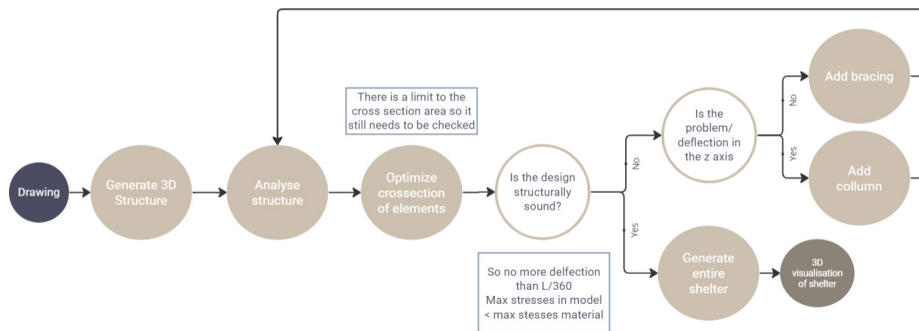
Figure 54 to 56: The generation of the columns and bracing in grasshopper
Source: Screen shots from rhino (parametrized design (3).gh)



5.5 Structural design

5.5.1 The idea

The idea for the structural implication in the project was that for each plan someone draws, an analysis would be made on the structure. From there we would have a list of crosssections of beams and columns and try to find the most optimal one for that design. Since they can't be infinitely big and the material will have a max span eventually extra columns will have to be added. To counteract the forces coming with the aftershocks extra bracing would also be needed. So when a optimal crosssection is found we see if and where bracing, or columns need to be added or removed.

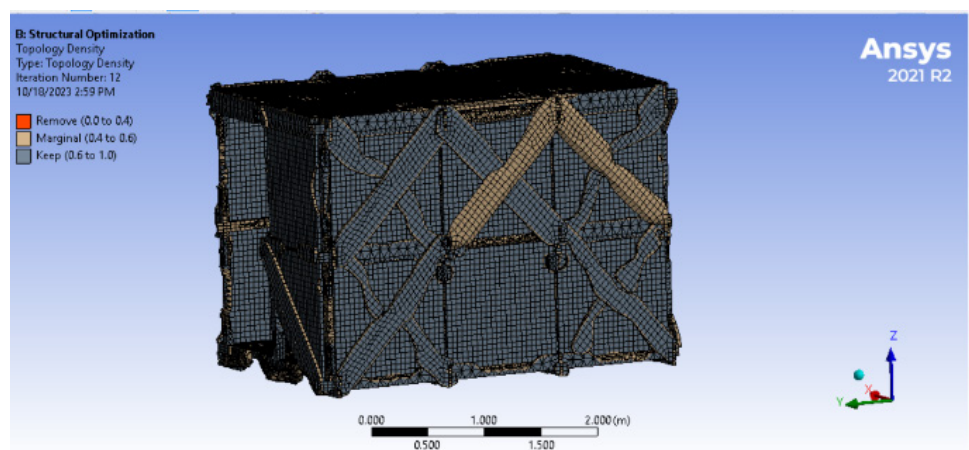


5.5.2 Our approach - Ansys

To get the structural analysis we at first tried out the Ansys workbench. Here you could load your rhino model into the software and make an analysis of the model. However, for each and every variation of the shelter you would have to by hand indicate what the supports joints and load were. It was able to fully recreate the structure and much more accurate in that way than the plugin for grasshopper we eventually used called Karamba 3D but it was just a lot slower and hard to automate.

We did do an analysis on one example of a shelter which was three by two and two meters high, one door, one window and bracing all around. With a variable load of 3kN/m^2 on the roof and floor, gravity, selfweight of the structure and an acceleration in one direction representing a movement that could happen during an earthquake. And with this method we were also able to optimize the structure and see where for example bracing wasn't necessary saving material. Which we didn't have yet in the Karamba 3D analysis in grasshopper but would like to add to it. For the other analysis see the appendix.

Figure 57: Ansys analysis material optimization
Source: Ansys workbench (own analysis)



5.5.3 Our approach - Karamba 3D

The ease of Karamba 3D was that we could use the same method for generating the structure as we were already adapting in rhino grasshopper and we could automatize the process of generating different options as well as different loads per option, and the assignment of type of joints, materials and crosssections of the members. Meaning for each alteration we would be able to maybe get a less accurate but still valid structural analysis within a quick timeframe.

So how did we go to work? In Karamba 3D you're unable to directly import the structure or geometries so we couldn't use the structure that the grasshopper python module would generate. Instead we first needed to create a simplified version of the structure for it to be able to work in Karamba (which is why it's less accurate than for example an ansys analysis).

We start off the same as with generating the actual structure. Basically getting the middle points from the CSV file points.csv and with a grasshopper python module create the same squares as drawn on the interface before.

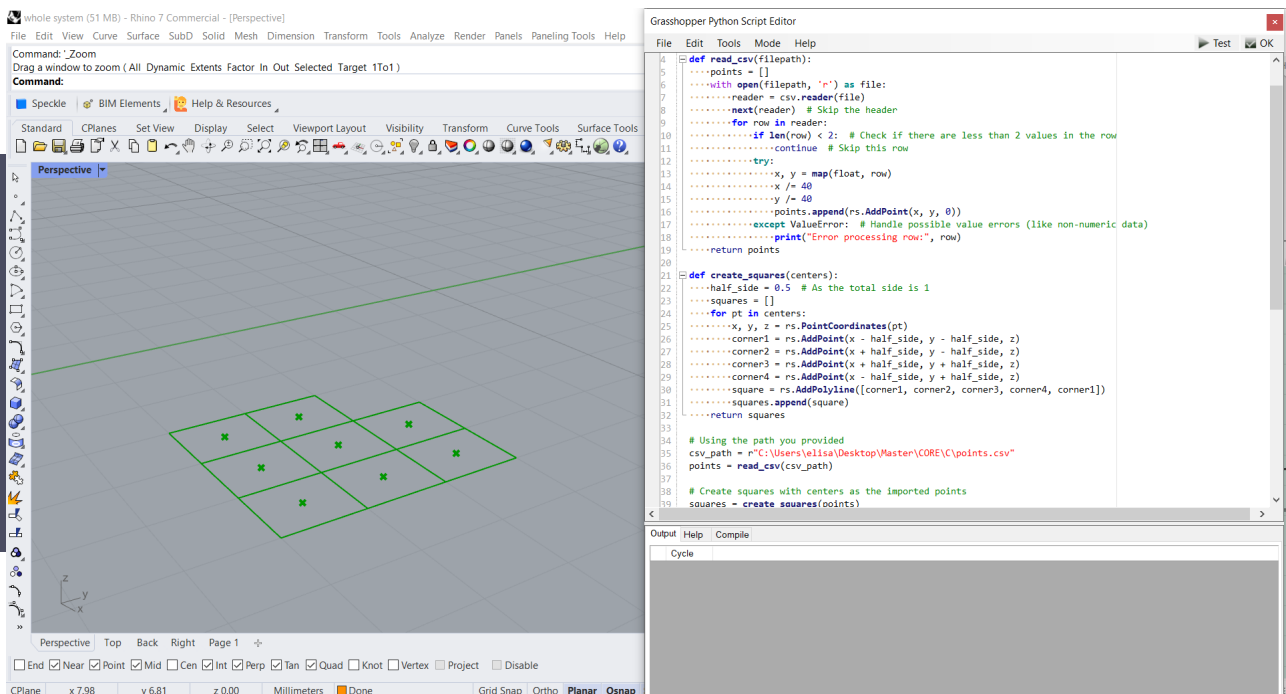


Figure 58: The grasshopper python script generating the plan
Source: Grasshopper rhino, own script

From there we continued by creating the entire grid for the shelter that would be fit to implement into the Karamba components. To create a model that the plugin would be able to read and analyse we needed a few things:

- ◊ Elem (elements): The beams and columns
- ◊ Support: The supporting joints that carry the loads to the foundation
- ◊ Load: The load that works on the structure
- ◊ Material: The material properties of the structure
- ◊ Joint: The joints/ connection points of two elements

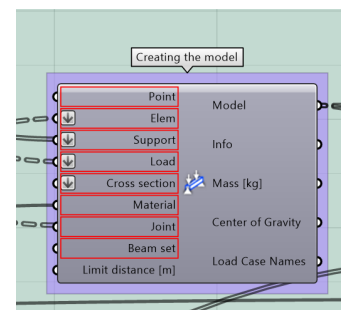


Figure 59: The module constructing the structure
Source: Karamba 3D, own script in grasshopper

The image shows a Grasshopper Python script on the left and a 3D visualization of a cube structure on the right. The script is titled 'Beams and columns' and is set within a purple background. It includes several components: a 'Materials' component with a dropdown menu showing 'Coniferous Timber' and a 'Color' dropdown showing 'C16(paralel)'; a 'Family' component with a dropdown menu showing 'C16(paralel)'; a 'Cross section' component with a dropdown menu showing 'Trapezoid'; a 'Panel' component with a list of values: 0.7, 5, 1.15, 2.30, 3.45; and an 'Options' component with a dropdown menu showing 'Line', 'Color', 'Identifier', 'Cross section', and 'Info'. The script is connected to a 3D visualization of a cube structure on the right, which is a 3D wireframe model of a cube with a grid of lines.

Figure 60: The grasshopper python script generating beam elements
Source: Grasshopper rhino, Karamba plugin, own script (togetherwesu.rvive.gh)

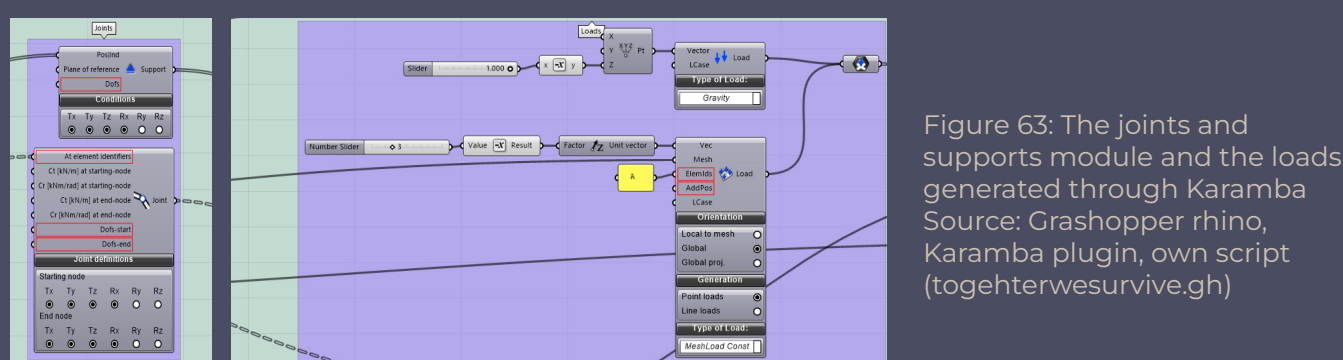
Figure 61: The grasshopper python script generating the plan

Source: Grasshopper rhino, own script (togetherwesurvive.gh)

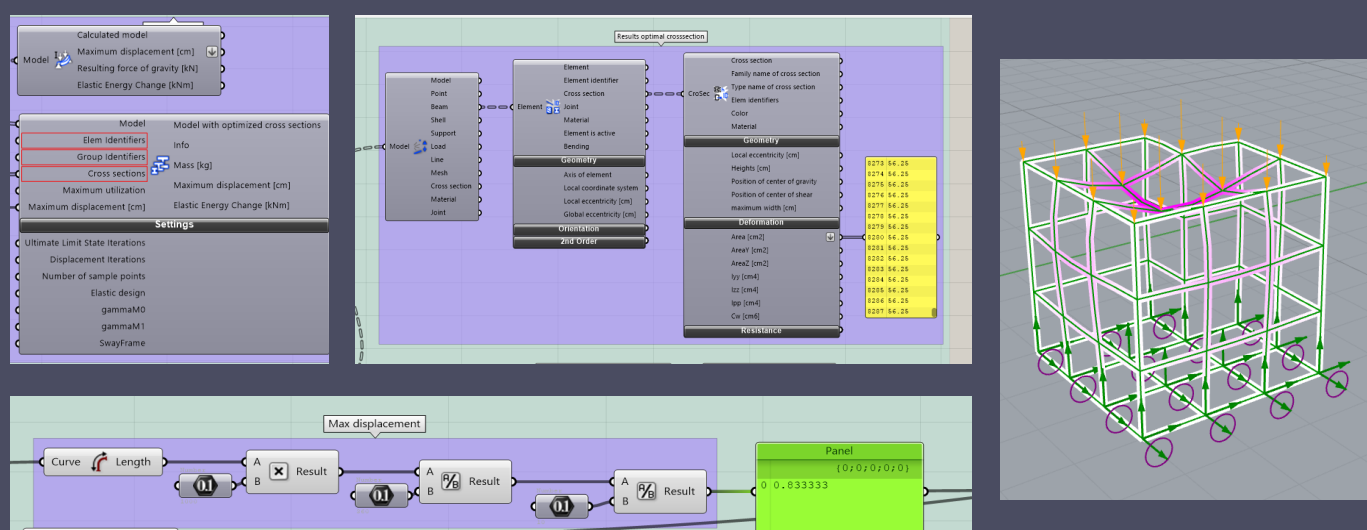
Figure 62: The grasshopper python script splitting the supports from the joints

The points are then connected to the module from Karamba creating the joints on the position of the points and the modules of freedom (conditions and joint definitions) of the joints are defined. In this case we have rigid joints and supports so Tx, Ty, Tz and Rx are ticked. T = translation/ movement of the joint (so unchecking Ty would mean free movement in the y direction) R = rotation or moment.

So lastly we would need to define the loads. In this case we add a load on top of the roof, and a force of gravity. In the future we would also like to add earthquake information to the component to create an acceleration load that would represent the movements of an earthquake so we can see how the structure would hold itself against that. We did already do this for one structure in ansys, but want to add this to Karamba as well to automate the process for each structure we generate.



From the model we run an optimization on the crosssection, using the rule of thumb that the deflection shouldn't be larger than $L/360$. Taking the largest span we calculate this value and set it as a max for the deflection. The Karamba component optimize cross section will then calculate what the best dimensions are for the beams and columns and give us a result like you see below. The results are the area of the crosssection, as well as stresses etc. can then be led back to the python script that generates the actual structure and depend their dimensions and orientations based on this analysis. Right now it only uses the dimensions of the beams and columns but in the future we would also like to add a feature that based on these calculations the structure would add columns and bracing accordingly.



5.6.2 A scale model

The scale model we created with it as you can see below here is fully made as it would in real life, so absolutely no glue, bolts, nails or any other fastening elements except for the friction between the elements.

While putting the model together we did find out a few interesting things to take into consideration.

- ◇ The shelter when building it is quite frail. Only when the elements are put together does the structure become rigid, especially after adding the braces, so maybe in reality we would need to provide some elements, tools or reinforcements for the loose elements so when building it up it is more rigid.
- ◇ One thing that we noticed was how if we put too much force on some of the elements of the joints they would break, so maybe in the future it would be good to also make a structural analysis, zooming in on the joint itself to see if the elements when loose would be able to hold certain loads.

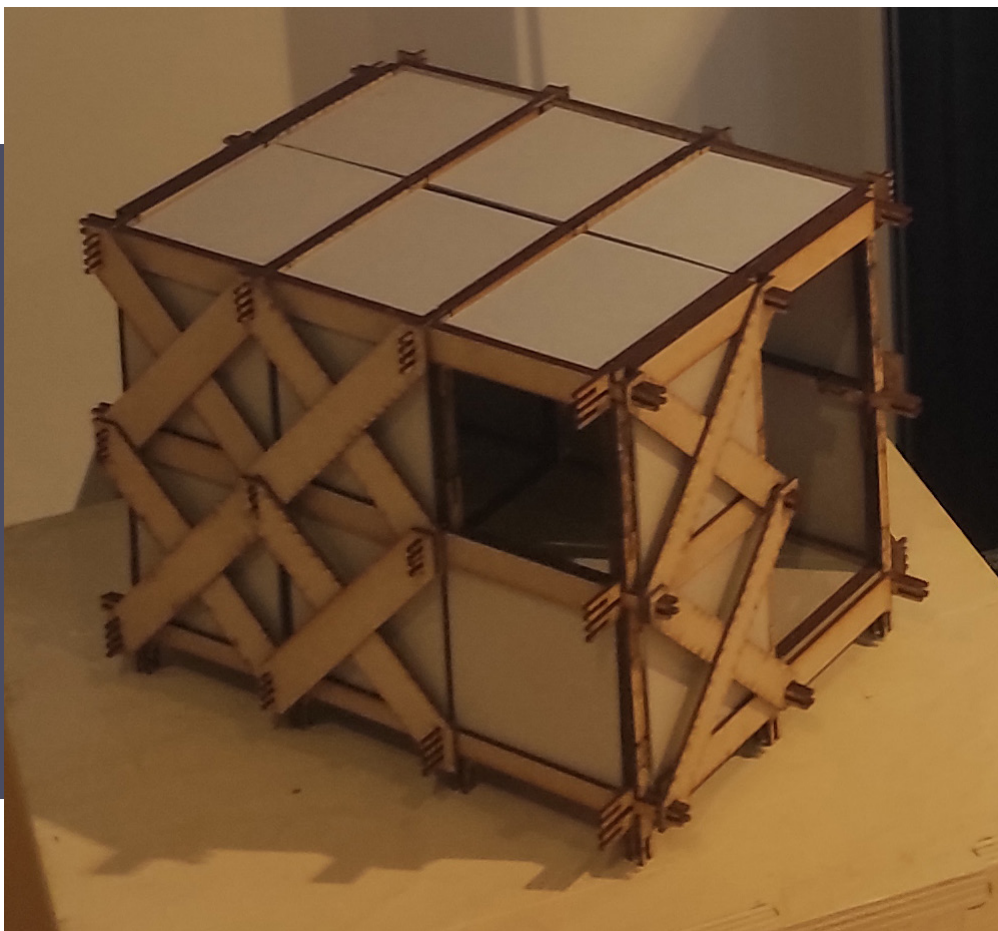


Figure 68: The line work created from the geometries by the command dup border and removing duplicate lines

Source: Grasshopper rhino, own script (see lazercut.gh)

6 Last thoughts

6.1 Conclusions

6.6.1 The final product

The idea for the structural implication in the project was that for each plan someone draws, an analysis would be made on the structure. From there we would have a list of cross sections of beams and columns and try to find the most optimal one for that design. Since they can't be infinitely big and the material will have a max span eventually extra columns will have to be added. To counteract the forces coming with the aftershocks extra bracing would also be needed. So when an optimal cross section is found we see if and where bracing, or columns needed to be added or removed.

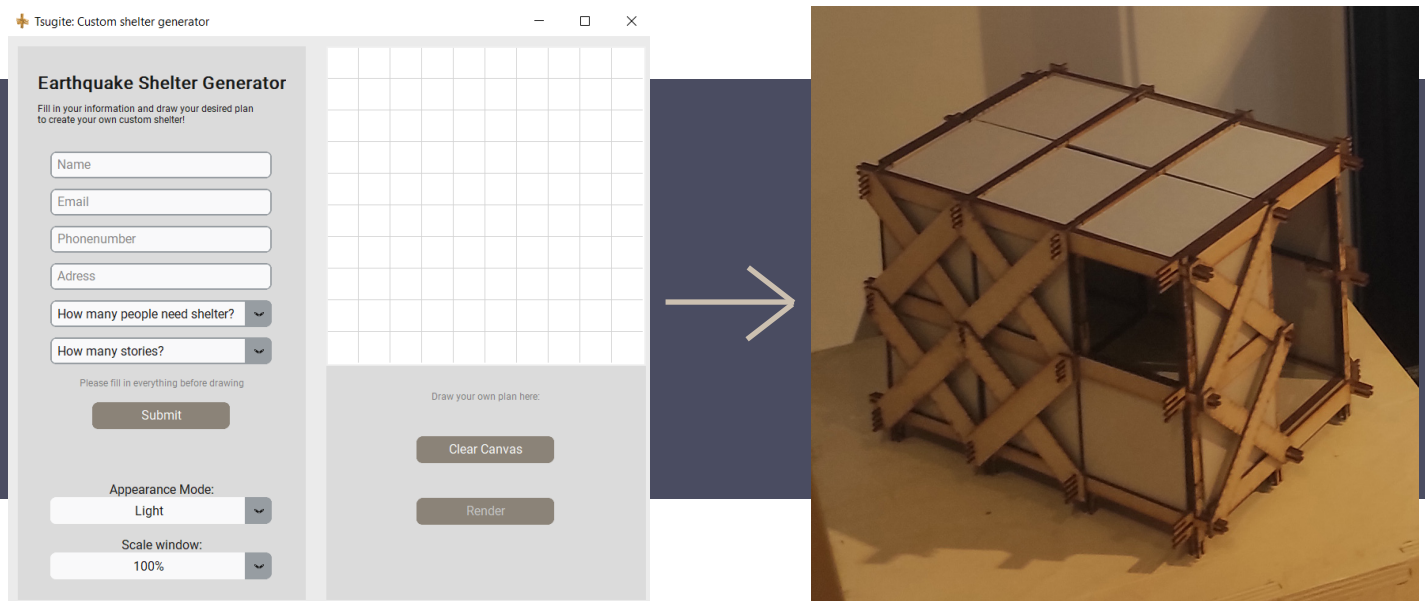


Figure 69 and 70: From interface to model Source: Own app and photo of model

6.6.2 Reflection

In hindsight looking back at the project the duo worked quite well and we were able to join our two works together as Elisa did most of the work for the interface and Amir for generating the structure in rhino, grasshopper. But we weren't shy to help each other out when stuck. One point of improvement might have been to have a more clear idea of how we would reach our goal and what tools would have been handy to use from the start. One example was the idea of using the tkinter GUI even though our initial idea was to create a website and along the way finding out these were not compatible. So starting with a flow chart instead of creating one later in the process. A plus point is that we were able to create a lot, and so also already got quite a nice result which of course a lot can be improved upon, but for the timeframe of little over a month we think we did quite well. And It was quite a journey to learn a new language basically and try to create something with the new toolkit created.

6.6.3 Future possibilities

As our project continues to evolve, we envision a series of enhancements and expansions that could further augment the efficacy and versatility of our earthquake-resistant structures.

These future developments include:

- ◇ Grid Size Variability: While our current model employs a uniform grid size of 1x1, we foresee the introduction of variable grid dimensions. This would offer greater flexibility in design and could be tailored to accommodate specific spatial requirements or aesthetic preferences.
- ◇ Dynamic Support Column Placement: Our present system designs supporting columns with a fixed span of 2x2. However, a more sophisticated approach could involve an optimization algorithm that determines the placement of support columns in real-time, based on areas experiencing high deflection, thereby enhancing structural integrity where it is most needed.
- ◇ Earthquake Resilience Simulation: To date, we have not incorporated earthquake resistance simulation into our design process. Future iterations could include comprehensive simulation and optimization for earthquake resilience, ensuring our structures not only stand but also provide the utmost safety during seismic events.
- ◇ Timber Use Optimization: While our current designs have not optimized for wood redundancy, we aim to integrate material optimization objectives in future models. By adjusting design parameters, we intend to minimize timber usage without compromising structural performance, contributing to environmental sustainability and cost-effectiveness.
- ◇ Material Diversification: Though we have focused on three types of timber for our initial designs, the potential to integrate alternative materials like steel presents an exciting avenue for exploration. This material diversification could result in structures with varied properties and new design potentials.
- ◇ Geometrical Diversity: The limitation to a single model geometry is something we plan to transcend in future projects. By embracing a wider array of structural geometries, we can cater to a broader range of architectural needs and preferences, possibly revolutionizing the spectrum of earthquake-resistant architecture.
- ◇ Architectural Feature Integration: The addition of architectural elements such as stairs, openings, and other functional components is a natural progression for our project. This would not only enhance the usability and livability of our structures **but also ensure that aesthetic considerations are given due weight alongside structural functionality.**

TSUGITE AS SHELTER

Home Contact Your Element orders



Figure 71: From interface to model Source: Own website
<https://amirrghadiril.wixsite.com/tsugite-as-shelter>

7 References

7.1 Literature

1 Shelter

- ♦ Kılçı, F., Kara, B. Y., & Bozkaya, B. (2015). Locating temporary shelter areas after an earthquake: A case for Turkey. *European Journal of Operational Research*, 243(1), 323–332. <https://doi.org/10.1016/j.ejor.2014.11.035>
- ♦ Kim, M., Kim, K., & Kim, E. (2021). Problems and Implications of Shelter Planning Focusing on Habitability: A Case Study of a Temporary Disaster Shelter after the Pohang Earthquake in South Korea. *International Journal of Environmental Research and Public Health*, 18(6), 2868. <https://doi.org/10.3390/ijerph18062868>
- ♦ Handbook, U. E. (2019, October 30). Emergency shelter standard. UNHCR. <https://emergency.unhcr.org/emergency-assistance/shelter-camp-and-settlement/shelter/emergency-shelter-standard#:~:text=Minimum%204.5m%20to%205.5,%2C%20eating%2C%20and%20livelihoods>

2 House hold size

- ♦ Statista. (2022, August 5). Average household size worldwide, by region 2019. <https://www.statista.com/statistics/1090668/average-household-size-worldwide-by-region/>
- ♦ UN - United Nations, Department of Economic and Social Affairs, Population Division (2019). Patterns and trends in household size and composition: Evidence from a United Nations dataset. (ST/ESA/SER.A/433).

3 Japanese joints

- ♦ Potvin, A., Demers, C., & Dumontier, C. (2013). Wooden Adaptive Architecture System (WAAS). *ResearchGate*. <https://doi.org/10.13140/2.1.3295.3924>
- ♦ Lily. (2018, August 8). Tsugite: The tetris of Japanese joinery | RETHINK Tokyo - Real Estate. *REthink Tokyo - Real Estate Information for Buyers and Investors*. <https://www.rethinktokyo.com/2018/08/08/tsugite-japanese-joinery-wood>
- ♦ ThemeGrill. (2021, January 27). japanese joinery techniques~ Tsugite) and Shiguchi) – Suikoushya-. <https://suikoushya.com/2021/01/27/tsugite-and-shiguchi/>
- ♦ King, V. (2021, March 3). GC Prostho Museum Research Center / Kengo Kuma & Associates. *ArchDaily*. <https://www.archdaily.com/199442/gc-prostho-museum-research-center-kengo-kuma-associates>
- ♦ Marialarsson. (2020). GitHub - marialarsson/tsugite: This is the repository of a Tsugite. It is a research prototype of an interactive software that supports the design and fabrication of wood joints. *GitHub*. <https://github.com/marialarsson/tsugite>

5 Himis

- ♦ Gülkan, P., & Langenbach, R. (2021). Traditional timber-laced masonry construction in Turkey known as himiş. In *Elsevier eBooks* (pp. 61–97). <https://doi.org/10.1016/b978-0-12-821087-1.00015-6>
- ♦ Dhajji Dewari. (n.d.). <https://www.world-housing.net/tutorials/other/dhajji-dewari>
- ♦ Usta, P., & Bozdağ, Ö. (2022). Seismic fragility analysis of traditional himis structures in Turkey. *Structures*, 43, 28–39. <https://doi.org/10.1016/j.istruc.2022.06.028>
- ♦ Ali, Q., Ahmad, N., Ashraf, M., Rashid, M. M., & Schacher, T. (2017). Shake table tests on Single-Story Dhajji Dewari traditional buildings. *International Journal of Architectural Heritage*, 1–14. <https://doi.org/10.1080/15583058.2017.1338789>

5 Materials

- ♦ Megan. (2023, March 20). Cost Engineering: Mass Timber vs. Concrete & Steel. *ASH*. <https://ash.com.au/blog/cost-engineering-mass-timber-vs-concrete-steel/>
- ♦ Homes, A. (2022, March 30). What's the average price to build a property in Turkey. *Antalya Homes - Tekce Overseas SA*. <https://antalyahomes.com/blog/market-analysis/what-s-the-average-price-to-build-a-property-in-turkey>
- ♦ Zengin, H., Yeşil, A., Asan, Ü., Bettinger, P., Cieszewski, C. J., & Siry, J. P. (2013). Evolution of modern forest management planning in the Republic of Turkey. *Journal of Forestry*, 111(4), 239–248. <https://doi.org/10.5849/jof.11-103>

7.2 Used libraries, plugins and other apps

1 Webservice:

- ◇ Gratis website maken | Maak je eigen website | Wix.com. (n.d.). NL - Wix HP Current. <https://nl.wix.com/>

2 Python libraries:

- ◇ Tkinter — Python interface to Tcl/Tk. (n.d.). Python Documentation. <https://docs.python.org/3/library/tkinter.html>
- ◇ TomSchimansky. (n.d.). GitHub - TomSchimansky/CustomTkinter: A modern and customizable python UI-library based on Tkinter. GitHub. <https://github.com/TomSchimansky/CustomTkinter>
- ◇ NumPy. (n.d.). <https://numpy.org/>
- ◇ re — Regular expression operations. (n.d.). Python Documentation. <https://docs.python.org/3/library/re.html>
- ◇ csv — CSV File Reading and Writing. (n.d.). Python Documentation. <https://docs.python.org/3/library/csv.html>
- ◇ Welcome to Flask — Flask Documentation (3.0.X). (n.d.). <https://flask.palletsprojects.com/en/3.0.x/>

3 Applications

- ◇ Ansys (2023, November 9). <https://www.ansys.com/>

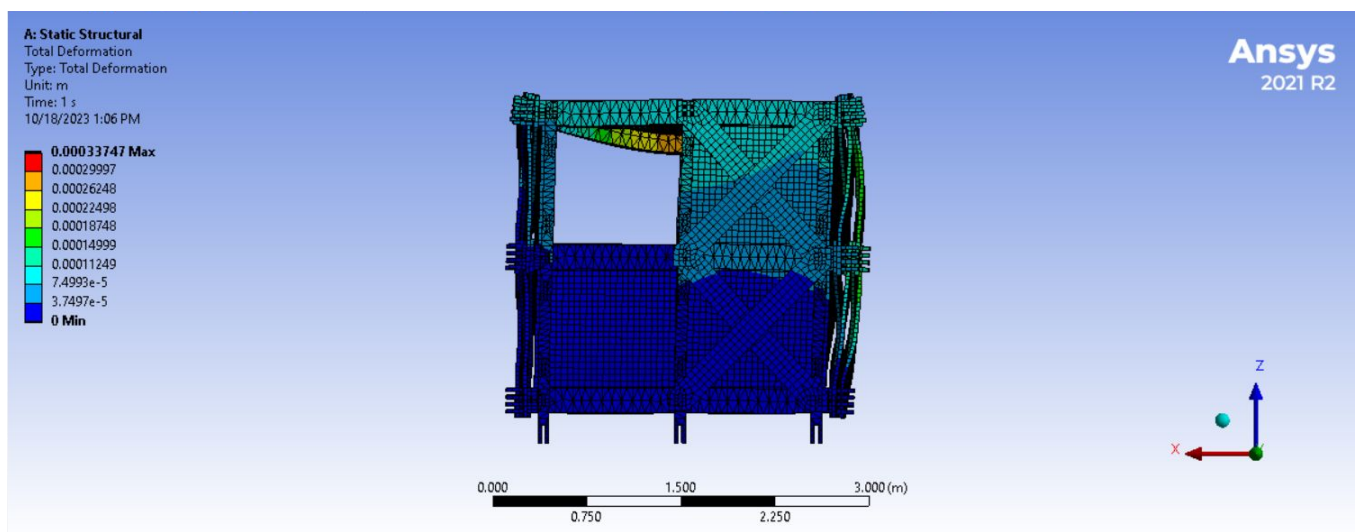
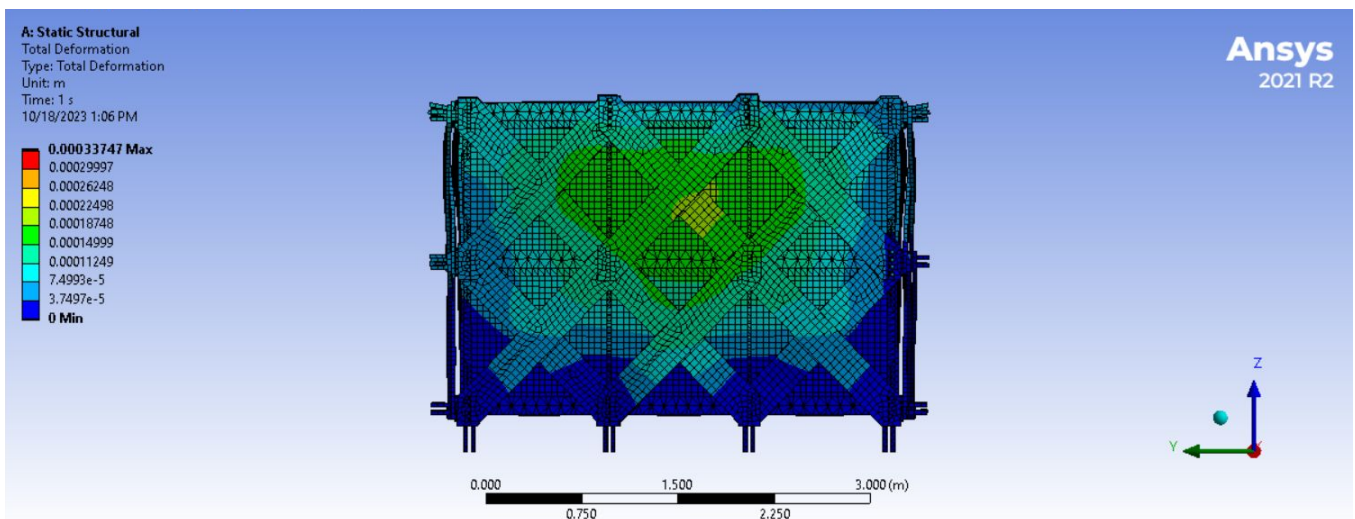
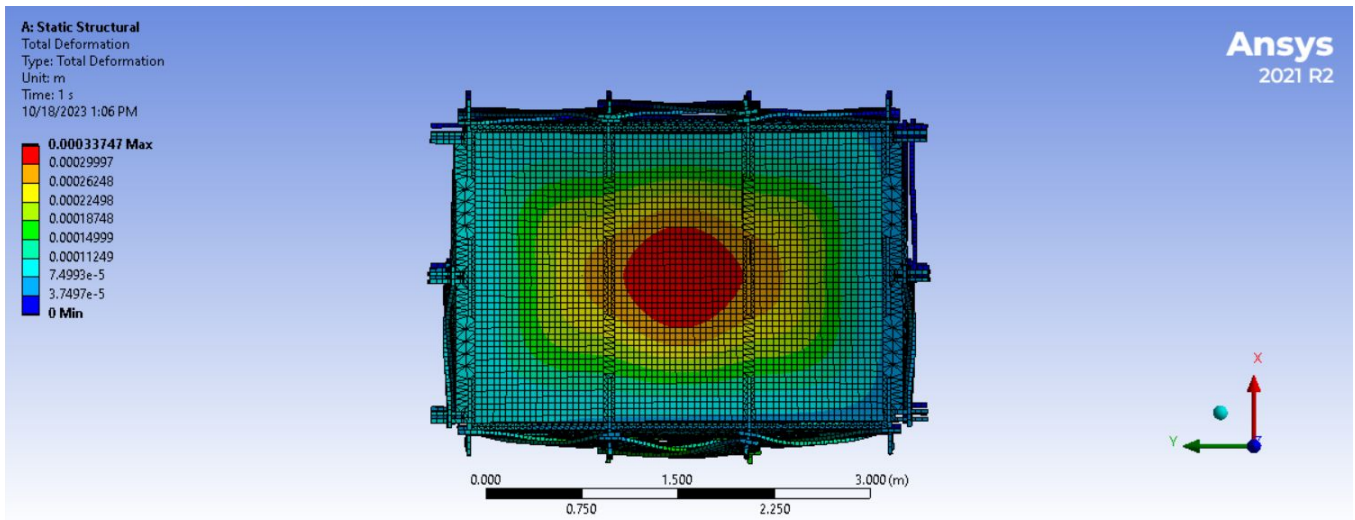
4 Plugins rhino grashopper

- ◇ Welcome to Karamba3D - Karamba3D 2.2.0. (n.d.). <https://manual.karamba3d.com/> and Karamba3D. (2023, May 25). Karamba3D. <https://karamba3d.com/>
- ◇ Robert McNeel & Associates. (n.d.). The Hops component. www.rhino3d.com. <https://developer.rhino3d.com/guides/compute/hops-component/>
- ◇ GHPython. (2022, May 17). Food4Rhino. <https://www.food4rhino.com/en/app/ghpython>

8 Appendix

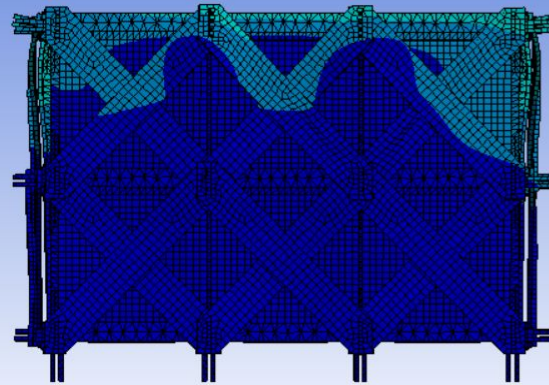
8.1 Analysis on a 2 by 3 shelter in Ansys

8.1.1 Deformation of an oakwood shelter, load floor 3 kN/m², gravity and acceleration:



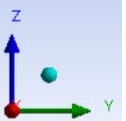
A: Static Structural
Total Deformation
Type: Total Deformation
Unit: m
Time: 1 s
10/18/2023 1:06 PM

0.00033747 Max
0.00029997
0.00026248
0.00022498
0.00018748
0.00014999
0.00011249
7.4993e-5
3.7497e-5
0 Min



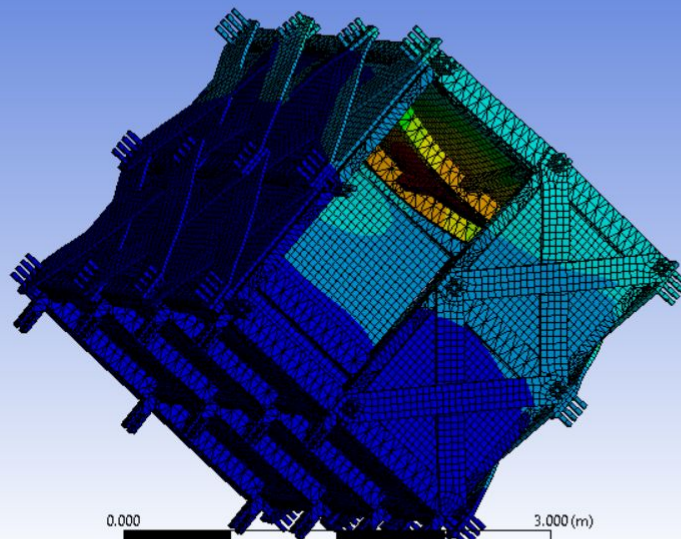
0.000 1.500 3.000 (m)
0.750 2.250

Ansys
2021 R2



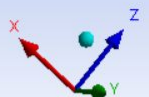
A: Static Structural
Total Deformation
Type: Total Deformation
Unit: m
Time: 1 s
10/18/2023 1:06 PM

0.00033747 Max
0.00029997
0.00026248
0.00022498
0.00018748
0.00014999
0.00011249
7.4993e-5
3.7497e-5
0 Min

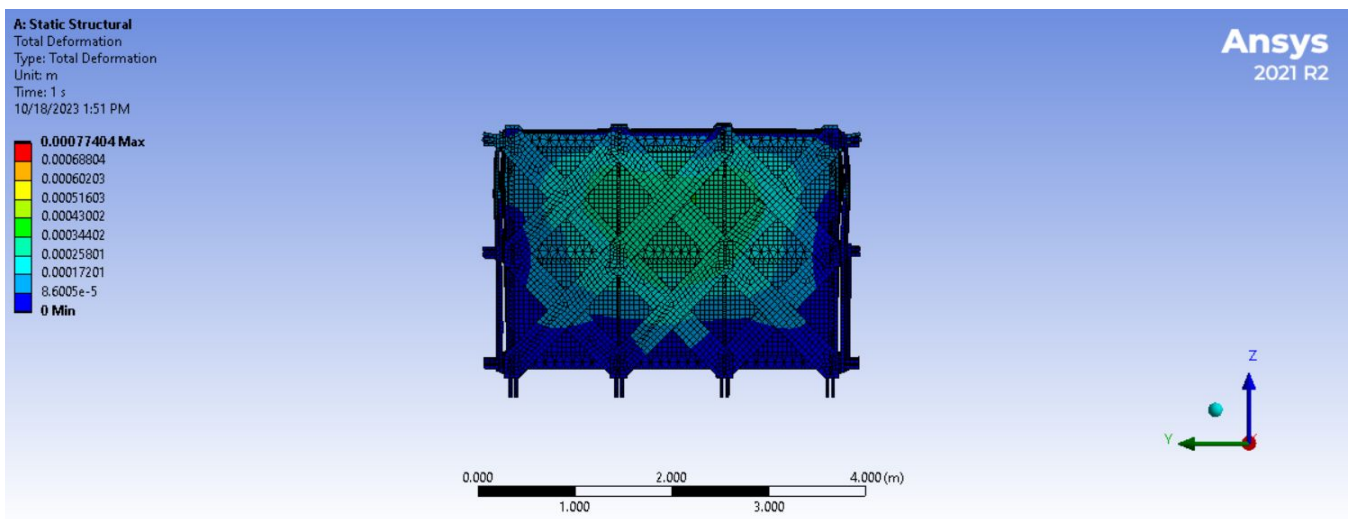
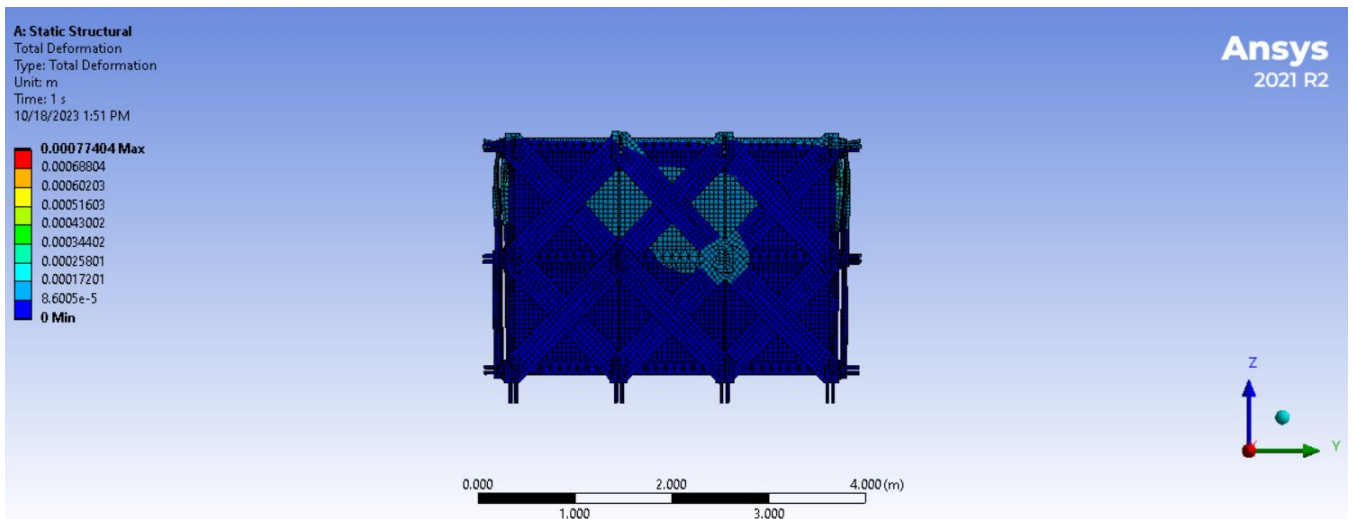
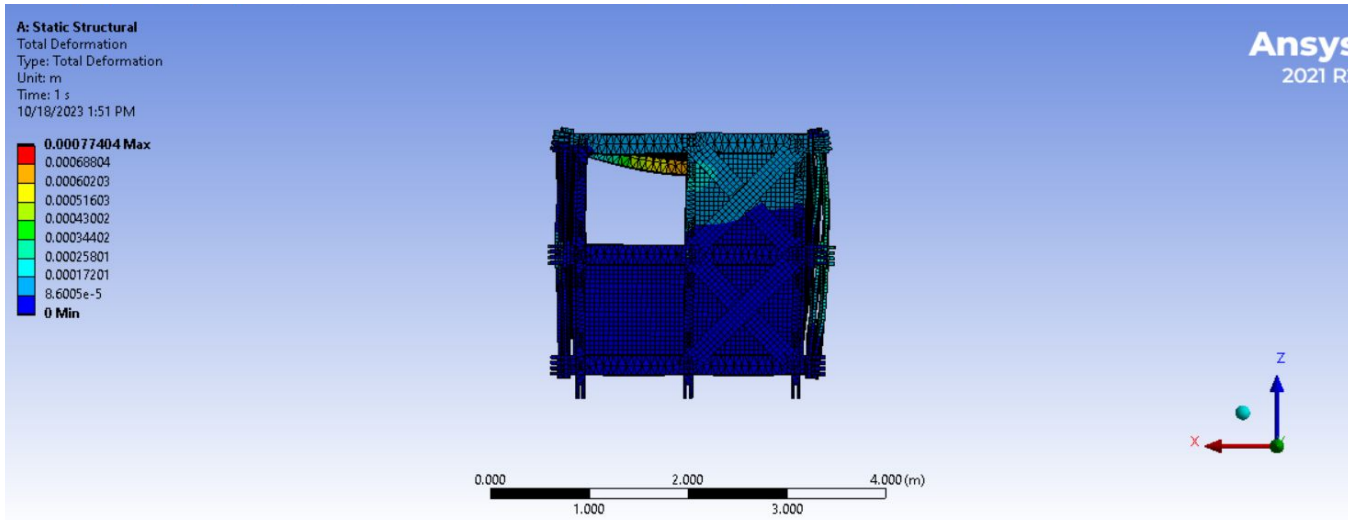


0.000 1.500 3.000 (m)
0.750 2.250

Ansys
2021 R2



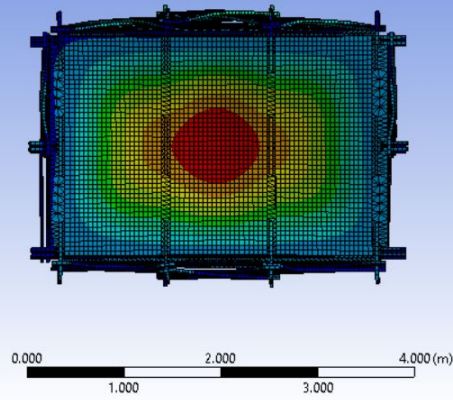
8.1.2 Deformation of a pinewood shelter, load floor 3 kN/m², gravity and acceleration:



A: Static Structural
Total Deformation
Type: Total Deformation
Unit: m
Time: 1 s
10/18/2023 1:51 PM

Ansys
2021 R2

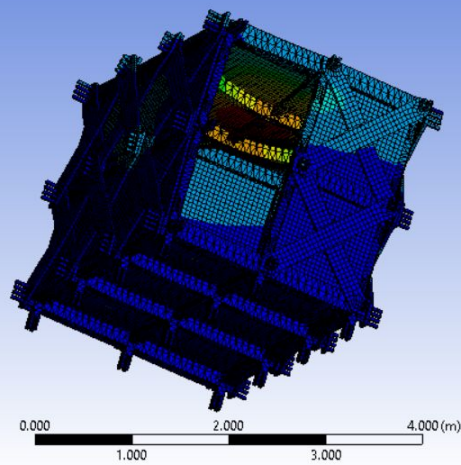
0.00077404 Max
0.00068804
0.00060203
0.00051603
0.00043002
0.00034402
0.00025801
0.00017201
8.6005e-5
0 Min



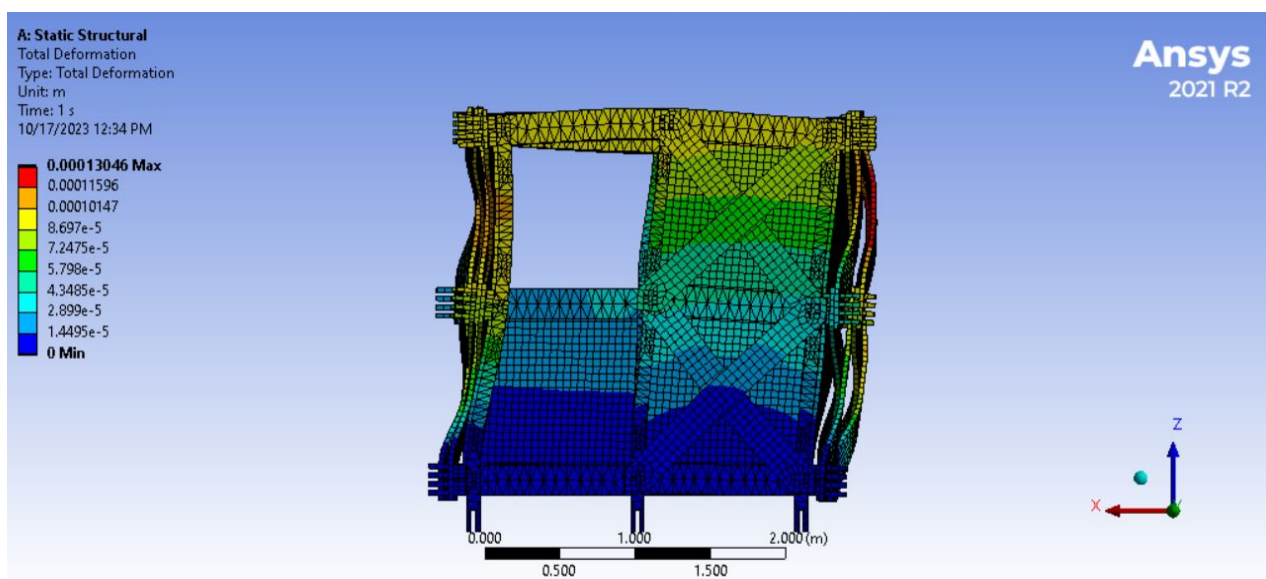
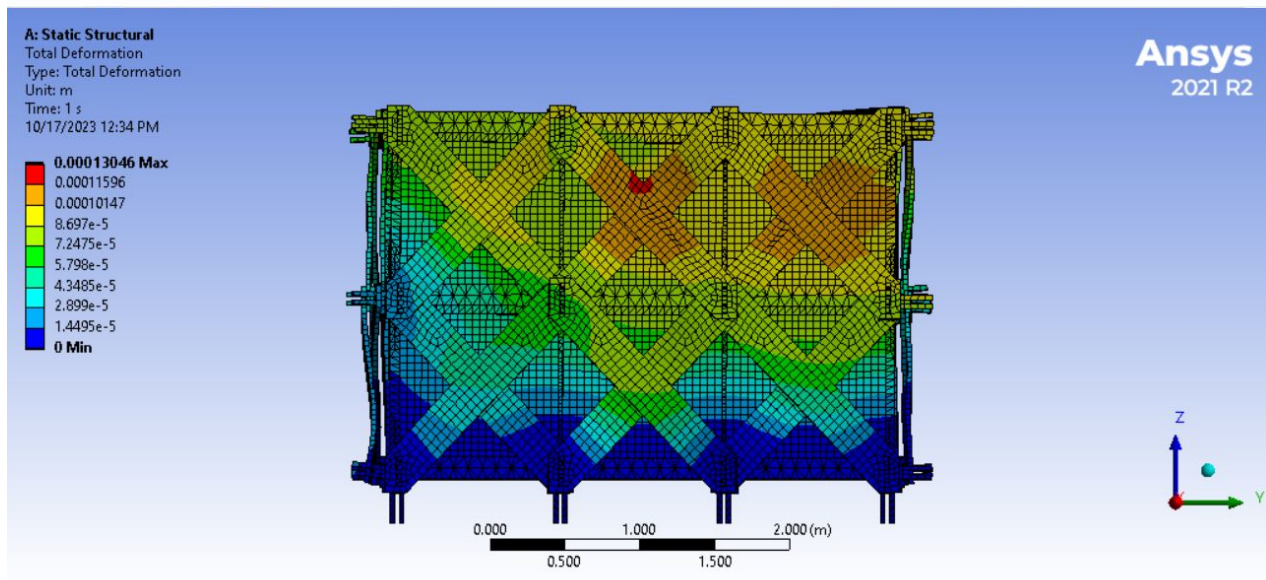
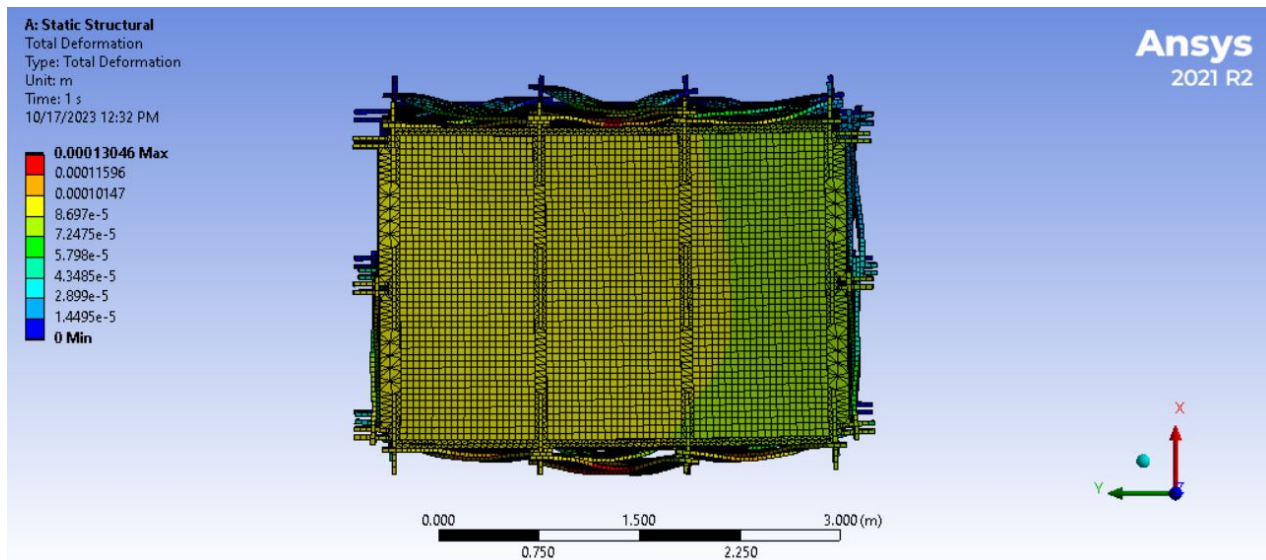
A: Static Structural
Total Deformation
Type: Total Deformation
Unit: m
Time: 1 s
10/18/2023 1:51 PM

Ansys
2021 R2

0.00077404 Max
0.00068804
0.00060203
0.00051603
0.00043002
0.00034402
0.00025801
0.00017201
8.6005e-5
0 Min

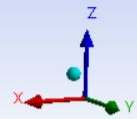
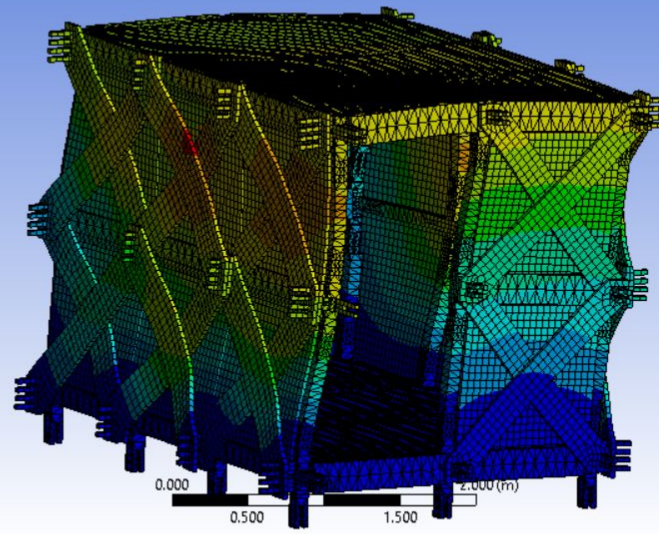
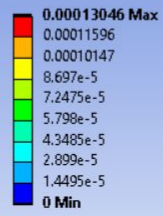


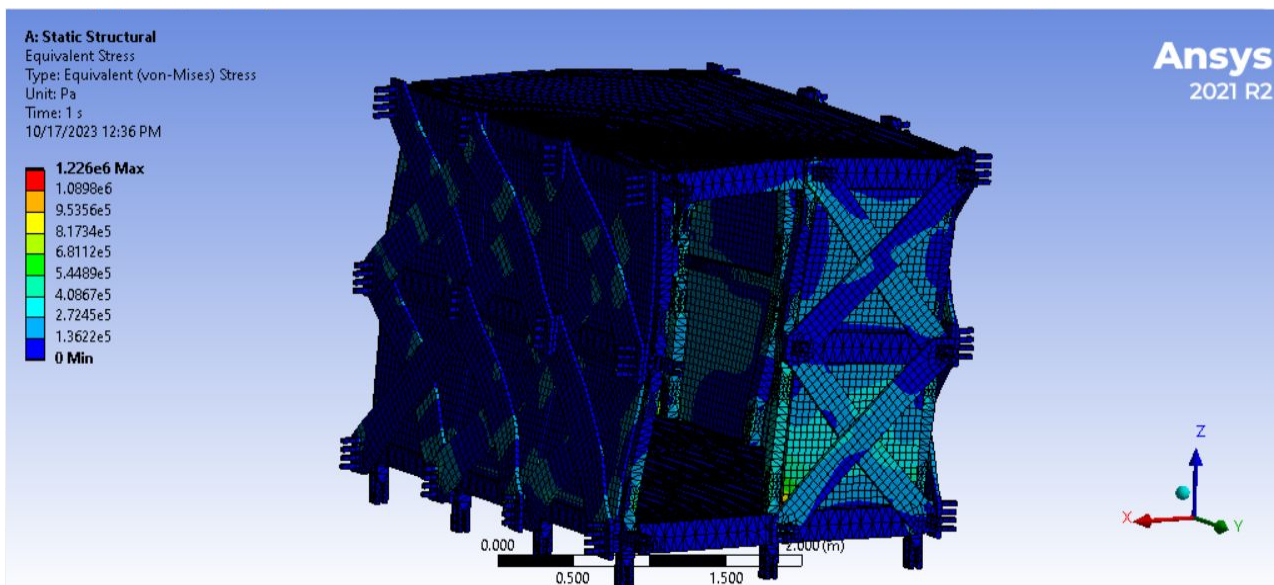
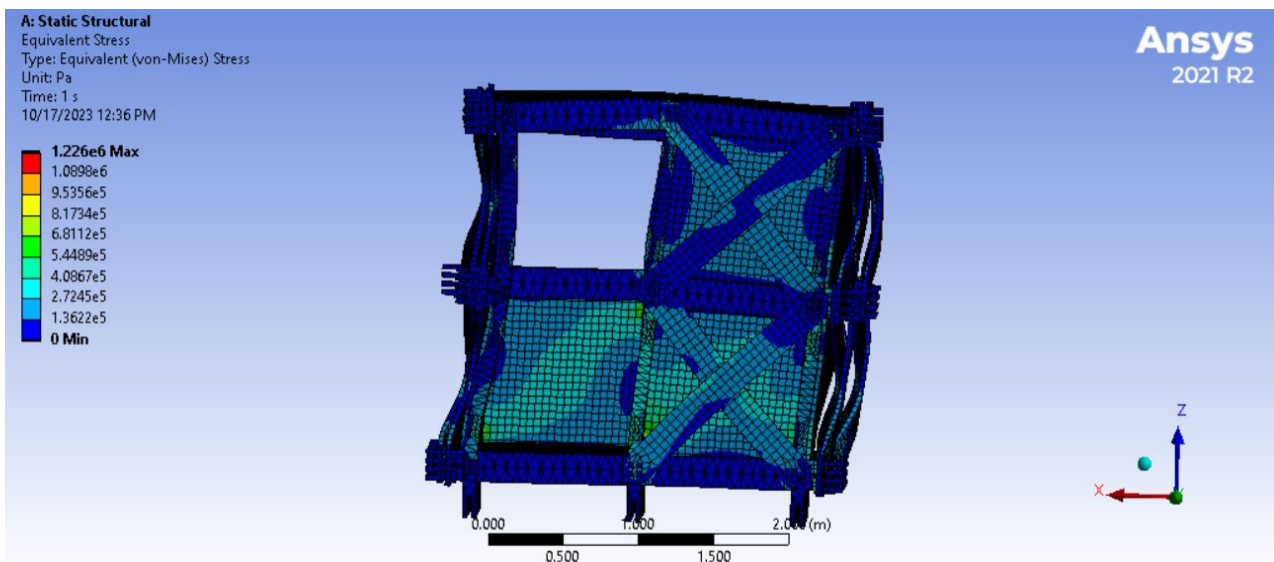
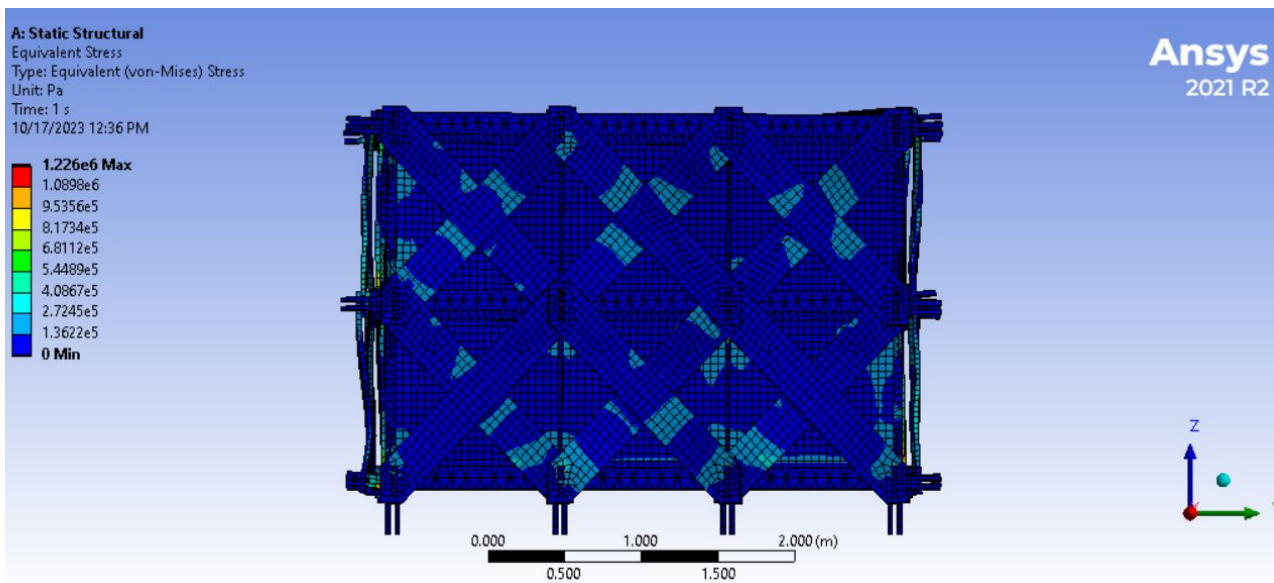
8.1.3 Analysis of a pinewood shelter, load floor 3 kN/m^2 , gravity and acceleration:



A: Static Structural
Total Deformation
Type: Total Deformation
Unit: m
Time: 1 s
10/17/2023 12:34 PM

Ansys
2021 R2





B: Structural Optimization

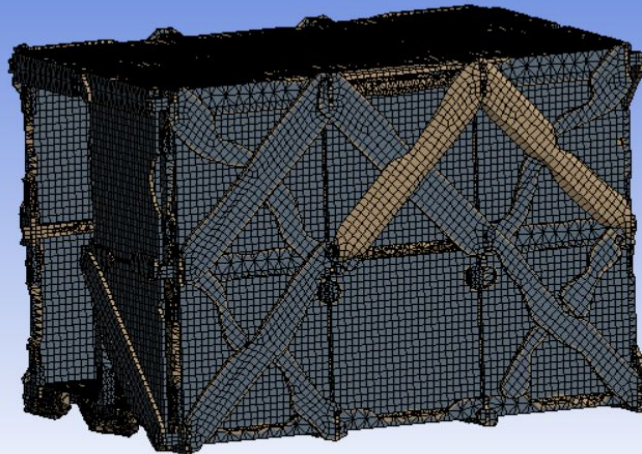
Topology Density

Type: Topology Density

Iteration Number: 12

10/18/2023 2:59 PM

- Remove (0.0 to 0.4)
- Marginal (0.4 to 0.6)
- Keep (0.6 to 1.0)

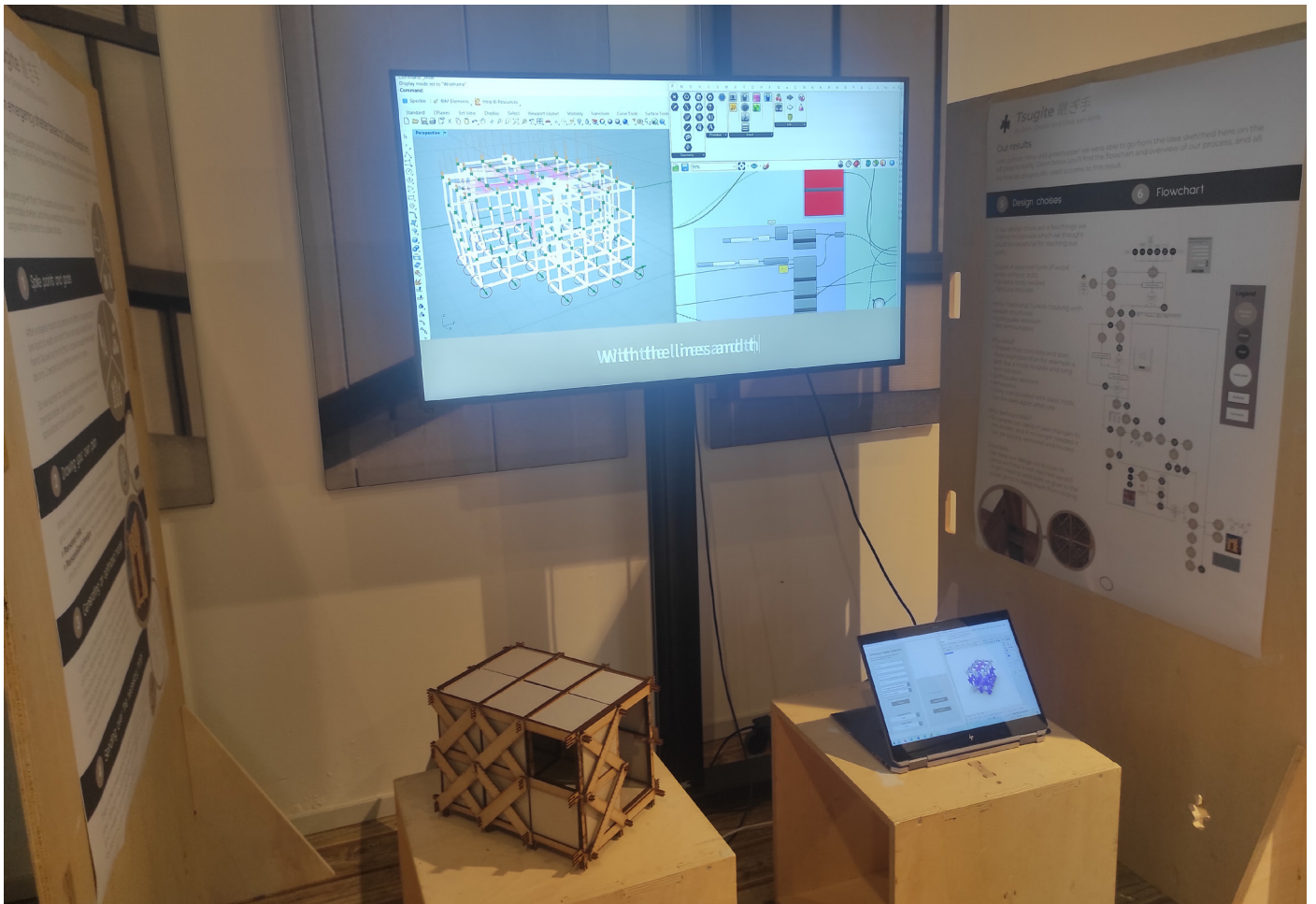


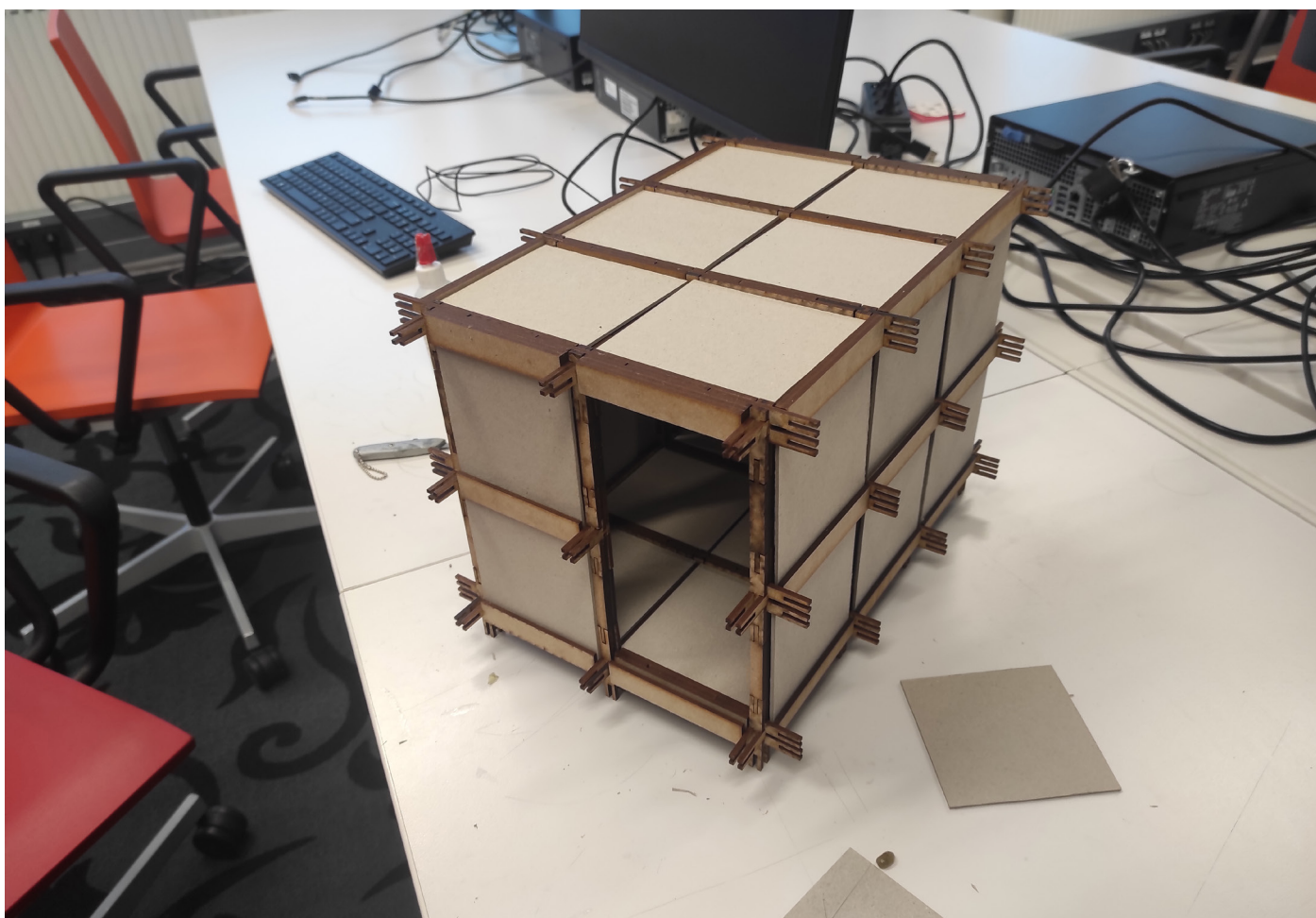
0.000 1.000 2.000 (m)
0.500 1.500

Ansys
2021 R2

8.2 Photos of the exhibition

8.2.1 Our own pictures





8.3 Posters



Tsugite 継ぎ手

By Amir Ghadiri and Elisa van Klink

Custom emergency shelter based on Japanese wood joints

After an earthquake people are in quick need of shelter. The first few days they are often housed in shabby tents in which they have to live in for the next few months or even years if the situation is dire.

We want to give them the opportunity to live in a more comfortable shelter until they are ready to find a new place or even upgrade their shelter to a place to stay.

1 Spike points and goals

After a disaster has struck, people are often in locations that are hard to reach, and won't have any specialized tools at hand. As well as the fact that in some houses there might be only 2 people but in others, even 6 or more.

So we wanted to create shelters that are light and easily transportable, quick and easy to build without much specialized tools and customizable.

2 Drawing your own plan

When you're in need of a shelter you can download our app and in the app you can leave your information:

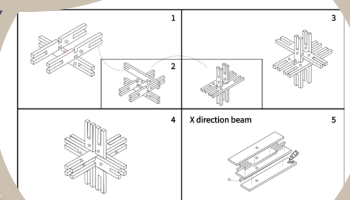
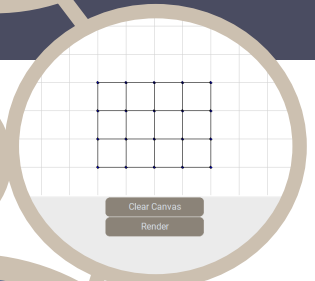
- ♦ **Personal info:** Name, email, phone number and location
- ♦ **Personalized design:** The amount of people needing shelter and the amount of floors and your desired floor plan.

3 Generating an optimized model

Looking at the design you created we run an analysis of the desired plan you came up with to create a 3D visualization, decide on size of the members and where possible extra columns need to be added. As well as decide where extra bracing is needed and where we can leave it out.

4 Sending over the necessary things

Then when everything is sorted out and you are happy with your design we will give you the instructions for assembly and make sure you get the parts needed to the correct location so you can start building!



Our results

With python, rhino and grasshopper we were able to go from the idea sketched here on the left page to reality. Down below you'll find the flowchart and overview of our process, and all the libraries, plugins etc. used to come to this result.

5

6

In our design there are a few things we tried to incorporate which we thought would be beneficial for reaching our goals.

Tsugite: A Japanese form of wood joinery without bolts

- + No extra tools needed
- Parts are intricate

Himis: Traditional Turkish housing with wooden structures

- + Earthquake resistant
- + Not demountable

Why wood?

- + Cheaper than concrete and steel, more expensive than for example a tent but a more durable and long term solution
- + Earthquake resistant
- + Renewable
- + Easily manipulated with basic tools
- + Can be used again after use

Why demountable?

- + So people can easily make changes to the shelter, and if no longer needed it can be quickly removed and reused

Downfalls:

- We base our design on friction fit joints so if this is not reached we still might need to add bolts or glue to the outer joints to keep them from sliding

